

WORKSHOP · 8 GIỜ · 5 SESSION

Kubernetes & Tự động hóa triển khai

Ebook workshop 8 giờ — Kubernetes core & CI/CD với Jenkins

Mario Education

Mục lục

PHẦN I · NỘI DUNG

1. Giới thiệu & Lộ trình học
2. Foundations & Cloud Landscape
3. Minikube, Pod, Deployment & YAML
4. Networking, Multi-env & RBAC
5. AI Deployment + Managed K8s (EKS/ECS)
6. CI/CD: GitHooks → Jenkins → Deploy

PHẦN II · BÀI GIẢI

1. Bài giải — Lab Session 2: Pod & Deployment
2. Bài giải — Lab Session 3: Networking & RBAC
3. Bài giải — Lab Session 4: Go Deployment 3 replicas + probes
4. Bài giải — Lab Session 5: Jenkins Pipeline & AI script

Phần I · Nội dung

Giới thiệu & Lộ trình học

Mở đầu Vì sao khóa học đi theo thứ tự này, và bạn sẽ đạt được gì sau 8 giờ.

Ebook này bám sát **syllabus workshop 8 giờ / 5 session**. Điểm khác biệt: mỗi bài không chỉ trả lời “*làm thế nào*” mà tập trung vào “**tại sao**”— vì hiểu lý do đằng sau một khái niệm mới là thứ giúp bạn tự xử lý được tình huống mới, thay vì học vẹt câu lệnh.

Bạn sẽ đạt được gì

Sau khi đọc hết và làm hết lab, bạn có thể:

- Giải thích được **tại sao cần Kubernetes** thay vì chạy container thủ công.
- Tự dựng cluster local bằng Minikube, deploy một ứng dụng FE + BE thật.
- Viết YAML cho Pod / Deployment / Service và hiểu từng dòng để làm gì.
- Phân quyền truy cập bằng **RBAC** theo môi trường (dev / staging).
- Phân biệt **K8s tự quản vs managed (EKS, ECS, GKE)** và biết khi nào chọn cái nào.
- Dựng một pipeline **CI/CD** hoàn chỉnh: commit → GitHooks → Jenkins → build → test → deploy.

Vì sao lộ trình đi theo thứ tự này

Mỗi session là một lớp nền cho session sau. Bỏ qua nền tảng sẽ khiến các phần sau trở thành “phép thuật”.

Session	Chủ đề	Vì sao đặt ở đây
1	Foundations + Cloud landscape	Phải hiểu <i>vấn đề</i> orchestration giải quyết trước khi học <i>công cụ</i> .
2	Minikube, Pod, Deployment, YAML	Bắt tay làm ngay trên máy local để biến lý thuyết thành trực giác.
3	Networking, Multi-env, RBAC	Ứng dụng thật cần nhiều thành phần nói chuyện với nhau — và cần kiểm soát ai được làm gì.
4	AI Deployment + Managed K8s	Chuẩn hóa deployment ở mức production, rồi mở rộng lên cloud.
5	CI/CD: GitHooks → Jenkins	Khép vòng: tự động hóa để mỗi commit tự build – test – deploy.

Cách dùng ebook

- **Nội dung** (menu trên): đọc lý thuyết theo thứ tự.
- **Bài giải:** lời giải chi tiết cho từng lab — nên tự làm trước rồi mới xem.
- **Reference:** nơi bạn tự thêm link ebook, video, tài liệu.

Chuẩn bị môi trường

Cài sẵn trước khi vào lab:

```
# Kiểm tra công cụ đã sẵn sàng
docker version
kubectl version --client
minikube version
git --version
```

Nếu cả bốn lệnh trên chạy được, bạn đã sẵn sàng cho Session 1.

Foundations & Cloud Landscape

90' Vì sao cần orchestration, kiến trúc K8s, và bức tranh cloud (EKS/ECS/GKE, Cloud Build).



Session 1 — Foundations & Cloud Landscape

Chào mừng bạn đến với buổi học đầu tiên. Trước khi gõ một dòng `kubectl` nào, chúng ta cần trả lời một câu hỏi quan trọng hơn nhiều: **tại sao Kubernetes lại tồn tại?** Nếu bạn hiểu được *vấn đề* mà nó giải quyết, mọi khái niệm phía sau (Pod, Deployment, Service, reconcile loop...) sẽ trở nên tự nhiên thay vì là một mớ thuật ngữ phải học thuộc.

Vì sao: Học công cụ mà không hiểu vấn đề gốc thì bạn chỉ “làm theo hướng dẫn”. Hiểu vấn đề gốc thì bạn có thể tự suy luận khi gặp tình huống mới.

1. Cuộc sống trước khi có orchestration

Giả sử bạn đã đóng gói ứng dụng thành một Docker image và chạy nó trên một server bằng tay:

```
docker run -d --name api -p 8080:8080 myapp:1.0
```

Với một container trên một máy, mọi thứ ổn. Nhưng thực tế production không dừng ở đó. Hãy xem những gì sẽ “vỡ”.

1.1. Container chết thì không tự dậy

Process trong container crash (hết RAM, bug, panic) → container dừng → ứng dụng của bạn *offline* cho đến khi có người phát hiện và chạy lại `docker run`. Lúc 3 giờ sáng thì ai làm việc đó?

Vì sao cần K8s: Ta muốn hệ thống **tự chữa lành** (self-healing) — cái gì chết thì tự khởi động lại, không cần con người canh.

1.2. Scale bằng tay rất cực và dễ sai

Traffic tăng, bạn cần 5 bản sao của API. Bạn SSH vào từng máy, chạy `docker run` 5 lần, rồi tự cấu hình load balancer trở tới 5 cổng khác nhau. Khi traffic giảm, lại SSH vào tắt bớt. Mỗi lần là một cơ hội gõ nhầm.

Vì sao cần K8s: Ta muốn nói “cho tôi 5 bản” và hệ thống tự lo phần còn lại — kể cả việc thay thế khi một bản chết.

1.3. Deploy phiên bản mới rất rủi ro

Muốn lên `myapp:2.0` ? Cách thủ công là tắt `v1.0` rồi bật `v2.0`. Giữa hai thao tác đó là **downtime**. Nếu `v2.0` lỗi, bạn phải nhớ chính xác cách quay lại `v1.0` — dưới áp lực, đêm khuya.

Vì sao cần K8s: Ta muốn deploy **cuốn chiếu** (rolling update) không downtime, và **rollback** chỉ bằng một lệnh.

1.4. Không biết cái gì chạy ở đâu

Có 10 server và 40 container. Container `payment` đang ở máy nào? Máy nào còn RAM để chạy thêm? Máy số 3 hỏng thì những gì đang chạy trên đó phải dời đi đâu? Quản lý bằng file Excel và trí nhớ là không khả thi.

Vì sao cần K8s: Ta muốn coi **cả cụm máy như một máy tính lớn (pool tài nguyên)**, và để phần mềm quyết định đặt container ở đâu.

Tóm lại: bài toán của orchestration

Vấn đề thủ công	Điều ta thực sự muốn
Container chết phải bật tay	Self-healing tự động
Scale bằng SSH + gõ lệnh	Khai báo số lượng, hệ thống tự đặt
Deploy có downtime, khó rollback	Rolling update, rollback 1 lệnh
Không biết gì chạy ở đâu	Scheduler tự xếp chỗ trên pool node
Cấu hình network/LB thủ công	Service discovery & load balancing sẵn có

Kubernetes (K8s) ra đời để giải quyết đúng danh sách này. Nó là một **container orchestrator**: hệ điều hành cho cả một cụm máy, chịu trách nhiệm chạy, giám sát, scale, và kết nối các container.

2. Ôn nhanh: Container vs VM

Vì K8s điều phối container, ta cần chắc chắn hiểu container khác VM ở đâu.

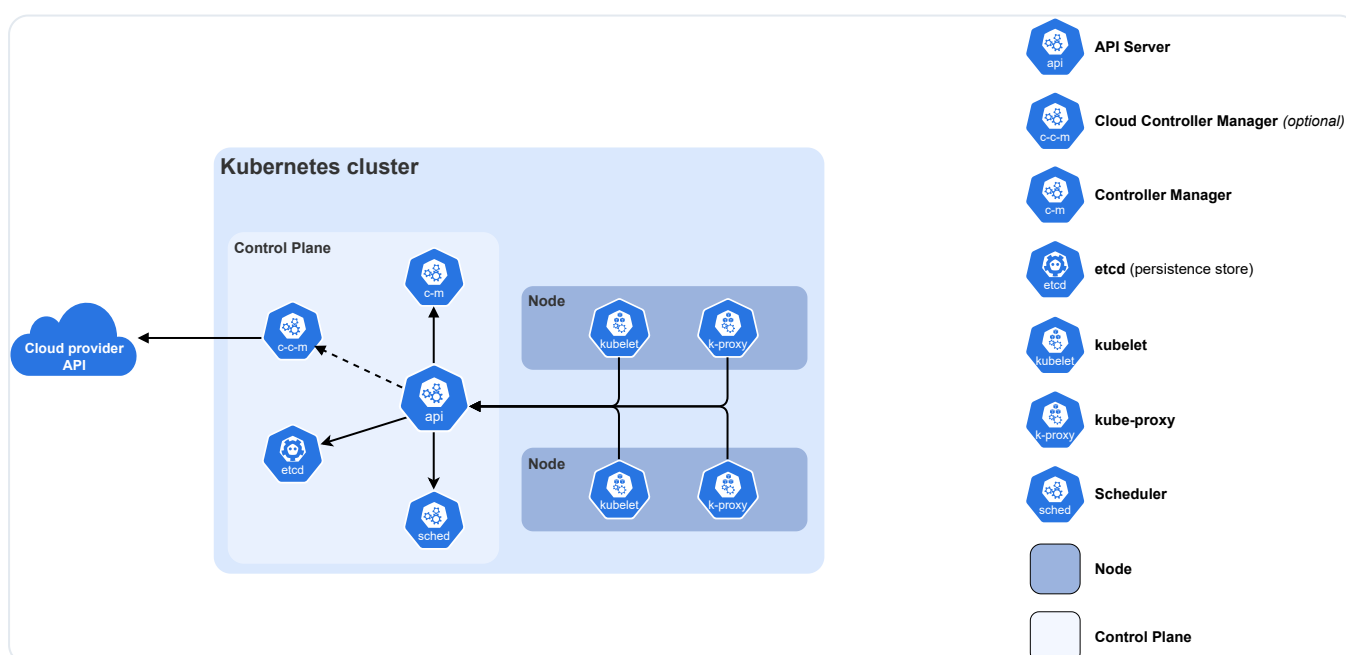
	Virtual Machine (VM)	Container
Ảo hóa ở tầng	Phần cứng (qua hypervisor)	Hệ điều hành (chia sẻ kernel)
Kèm theo	Cả một OS guest đầy đủ	Chỉ app + thư viện cần thiết
Kích thước	Hàng GB	Thường hàng chục–vài trăm MB
Thời gian khởi động	Vài chục giây–phút	Mili giây đến vài giây
Mức cô lập	Rất mạnh (OS riêng)	Nhẹ hơn (chung kernel)

Vì sao container hợp với orchestration: Vì nhẹ và khởi động nhanh, ta có thể tạo/hủy hàng loạt bản sao trong tích tắc. Scale và self-healing chỉ khả thi khi việc “sinh ra một bản mới” rẻ và nhanh — điều VM không làm tốt.

Container **không thay thế** VM: trong thực tế, các node K8s thường chính là các VM trên cloud. Ta có container *bên trong* VM.

3. Kiến trúc Kubernetes

Một K8s **cluster** gồm hai loại thành phần: **Control Plane** (bộ não, ra quyết định) và **Worker Node** (nơi thực sự chạy container).



3.1. Control Plane — bộ não của cụm

Control Plane không chạy ứng dụng của bạn; nó quản lý trạng thái toàn cụm. Nó gồm nhiều thành phần được **tách rời có chủ đích**:

Thành phần	Vai trò	Tại sao tách riêng
API Server	Cổng vào duy nhất; nhận mọi lệnh (<code>kubectl</code> , controller, node) qua REST	Một điểm vào duy nhất giúp tập trung xác thực, phân quyền, kiểm tra hợp lệ. Mọi thành phần khác chỉ nói chuyện qua API Server, không nói trực tiếp với nhau.
etcd	Cơ sở dữ liệu key-value, lưu toàn bộ trạng thái cụm	Tách phần “lưu trữ sự thật” ra riêng, bền vững và nhất quán. API Server không tự giữ state — nó đọc/ghi vào etcd. Mất etcd = mất trí nhớ cụm.
Scheduler	Quyết định Pod mới nên chạy trên node nào	Việc “xếp chỗ” là một bài toán riêng (dựa trên CPU/RAM còn trống, ràng buộc, ái lực). Tách ra để logic này thay thế/nâng cấp độc lập.
Controller Manager	Chạy các vòng lặp điều khiển: so trạng thái mong muốn với thực tế và sửa cho khớp	Đây là nơi hiện thực “self-healing”. Tách riêng để mỗi loại controller (Deployment, Node, Job...) làm đúng một việc.

Vì sao kiến trúc “mọi thứ đi qua API Server”: Nó biến API Server thành *nguồn sự thật giao tiếp* duy nhất. Nhờ vậy hệ thống dễ mở rộng — muốn thêm tính năng, bạn chỉ cần viết một controller mới lắng nghe API Server, không phải sửa lỗi.

Ngoài ra còn **cloud-controller-manager** (trên môi trường cloud) để nói chuyện với hạ tầng nhà cung cấp (tạo load balancer, disk...).

3.2. Worker Node — nơi công việc thực sự diễn ra

Mỗi worker node chạy ba thành phần chính:

Thành phần	Vai trò
kubelet	“Đại diện” của K8s trên node. Nhận lệnh từ Control Plane, đảm bảo các container trong Pod đang chạy đúng và báo cáo tình trạng về.
kube-proxy	Lo phần mạng trên node: định tuyến traffic tới đúng Pod, hiện thực Service.
Container runtime	Phần mềm thực sự chạy container (ví dụ containerd, CRI-O). kubelet ra lệnh, runtime thực thi.

Vì sao tách kubelet khỏi runtime: K8s không tự chạy container mà ủy thác cho runtime qua một chuẩn chung (CRI). Nhờ đó bạn có thể đổi runtime mà không phải viết lại K8s.

4. Các đối tượng cốt lõi và quan hệ

Đây là chuỗi trừu tượng từ lớn đến nhỏ:

```
Cluster → Node → Pod → (được quản lý bởi) ReplicaSet → (được quản lý bởi) Deployment
                        ↑
                Service (địa chỉ mạng ổn định trỏ tới nhóm Pod)
```

4.1. Pod — đơn vị nhỏ nhất (không phải container)

Đây là điểm gây nhầm lẫn nhất cho người mới. K8s **không trực tiếp quản lý container**, nó quản lý **Pod**. Một Pod là một hoặc nhiều container luôn được xếp cùng nhau, chia sẻ chung network (cùng IP) và có thể chia sẻ volume.

Vì sao Pod là đơn vị nhỏ nhất chứ không phải container: Có những container “sinh đôi” phải sống cùng nhau — ví dụ một app chính và một “sidecar” đẩy log. Chúng cần chung IP, cần được đặt cùng node, cần sinh/tử cùng nhau. Pod là cái “hộp” đảm bảo điều đó. Đa số Pod chỉ có 1 container, nhưng khái niệm Pod cho phép mô hình sidecar khi cần.

Pod là **phù du (ephemeral)**: nó có thể bị xóa và thay bằng Pod mới (IP mới) bất cứ lúc nào. Đây là lý do ta cần các lớp bên trên.

4.2. ReplicaSet — giữ đúng số lượng Pod

ReplicaSet đảm bảo luôn có đúng N bản sao Pod đang chạy. Một Pod chết? ReplicaSet tạo Pod mới thay thế. Đây chính là nơi "self-healing" và "scale" thành hình.

4.3. Deployment — quản lý ReplicaSet qua thời gian

Bạn hầu như không tạo ReplicaSet trực tiếp. Bạn tạo **Deployment**, và nó quản lý ReplicaSet giúp bạn.

Vì sao cần thêm một lớp Deployment: ReplicaSet giỏi việc "giữ N bản", nhưng không lo việc *nâng cấp phiên bản*. Deployment thêm khả năng rolling update và rollback: khi bạn đổi image sang `v2.0`, Deployment tạo ReplicaSet mới, tăng dần Pod mới và giảm dần Pod cũ — không downtime. Lỗi thì `kubectl rollout undo`.

4.4. Service — địa chỉ ổn định giữa những Pod phù du

Vì Pod hay bị thay và đổi IP, ta không thể để client nhớ IP của từng Pod. **Service** cung cấp một địa chỉ (tên DNS + IP ảo) *cố định*, tự động cân bằng tải tới các Pod đang khỏe phía sau.

Vì sao cần Service: Nó giải bài toán "làm sao gọi được một thứ luôn thay đổi địa chỉ". Service là điểm neo ổn định; các Pod đứng sau có thể sinh tử tùy ý.

5. Mô hình Declarative & Reconcile Loop — trái tim của K8s

Đây là ý tưởng quan trọng nhất buổi hôm nay. Hãy hiểu nó thật kỹ.

Có hai cách ra lệnh cho hệ thống:

- **Imperative (mệnh lệnh):** bạn ra từng bước — "tạo Pod này", "xóa Pod kia", "khởi động lại cái đó". Bạn chịu trách nhiệm cho *cách* đạt kết quả.
- **Declarative (khai báo):** bạn mô tả **trạng thái mong muốn** (desired state) — "tôi muốn 3 bản của `myapp:2.0`" — và để hệ thống tự tìm cách đạt được.

Kubernetes theo mô hình **declarative**. Bạn viết một file YAML mô tả điều mình muốn:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: myapp
spec:
  replicas: 3
  selector:
    matchLabels:
      app: myapp
  template:
    metadata:
      labels:
        app: myapp
    spec:
      containers:
        - name: myapp
          image: myapp:2.0
          ports:
            - containerPort: 8080
```

Rồi bạn “gửi ước muốn” đó cho cụm:

```
kubectl apply -f deployment.yaml
```

Reconcile loop hoạt động thế nào

Các controller trong Control Plane liên tục chạy một vòng lặp:

Lặp mãi mãi:

1. Đọc DESIRED STATE (trạng thái mong muốn, lưu trong etcd)
2. Quan sát ACTUAL STATE (thực tế đang chạy)
3. Nếu khác nhau → hành động để kéo thực tế về gần mong muốn

Ví dụ: bạn khai báo `replicas: 3` nhưng chỉ có 2 Pod sống. Chênh lệch! Controller tạo thêm 1 Pod. Một node chết làm mất 1 Pod → lại chênh → lại tạo bù. Bạn không hề can thiệp.

Vì sao mô hình này *mạnh*: Bạn chỉ cần khai báo “đích đến” một lần. Hệ thống *tự* duy trì đích đó mãi mãi, chống lại mọi biến cố (crash, node hỏng, xóa nhầm). Đây chính là gốc rễ của self-healing. So với imperative — nơi bạn phải liên tục ra lệnh và tự phát hiện lỗi — declarative giao gánh nặng đó cho máy.

Vì sao YAML nên được lưu trong Git: Vì file khai báo *chính* là nguồn sự thật, đưa nó vào Git cho bạn lịch sử thay đổi, review, và khả năng tái tạo cụm. Đây là nền tảng của “GitOps” mà ta sẽ gặp ở các buổi sau.

6. Bức tranh Cloud: chọn chạy K8s ở đâu

Bạn có thể tự dựng K8s, hoặc dùng bản “managed” của nhà cung cấp cloud. Và có cả những orchestrator *không phải* K8s.

6.1. Các lựa chọn chính

Dịch vụ	Là gì	Ai vận hành Control Plane
Minikube / kubeadm	K8s tự cài (học tập / tự quản)	Bạn
EKS (AWS)	Managed Kubernetes trên AWS	AWS lo Control Plane
GKE (Google)	Managed Kubernetes trên Google Cloud	Google lo Control Plane
ECS (AWS)	Orchestrator riêng của AWS , KHÔNG phải Kubernetes	AWS
Cloud Build (Google)	Dịch vụ CI/CD serverless: build image → đẩy lên registry	Google (không cần server)

Vì sao “managed” tồn tại: Tự vận hành Control Plane (etcd, API Server, vá lỗi, backup, HA) rất tốn công và dễ sai. EKS/GKE nhận phần khó đó, để bạn tập trung vào ứng dụng. Đổi lại bạn trả thêm tiền và mất một chút quyền kiểm soát tầng thấp.

6.2. ECS — điểm dễ nhầm

ECS (Elastic Container Service) KHÔNG phải Kubernetes. Đây là orchestrator do AWS tự thiết kế, dùng khái niệm riêng (Task, Task Definition, Service) thay vì Pod/Deployment.

Vì sao chọn ECS thay vì EKS: Nếu bạn *chỉ* dùng AWS, muốn đơn giản, ít khái niệm, tích hợp sâu với hệ sinh thái AWS và không cần sự linh hoạt/đa đám mây của K8s — ECS nhẹ đầu hơn. Đặc biệt với ECS Fargate, bạn còn không phải quản lý node. Ngược lại, chọn **EKS** khi bạn cần chuẩn K8s (kỹ năng, công cụ, Helm, portable giữa các cloud) hoặc đội ngũ đã quen K8s.

6.3. Google Cloud Build — mảnh ghép CI/CD

Cloud Build không chạy ứng dụng của bạn lâu dài; nó là **serverless CI/CD**: nhận source code, build Docker image theo các bước bạn khai báo, rồi đẩy image lên registry để cluster (GKE/EKS...) kéo về chạy.

Vì sao tách build ra khỏi cluster: Việc build image là ngắn hạn và tốn tài nguyên theo đợt. Dùng dịch vụ serverless chỉ trả tiền khi build, thay vì nuôi server build chạy suốt. Đây là bước "CI" nối liền code với "CD" trên K8s — chủ đề trọng tâm các buổi sau.

6.4. Tự quản vs Managed — chọn thế nào

Tiêu chí	Tự quản (Minikube/kubeadm)	Managed (EKS/GKE)
Chi phí tiền mặt	Thấp (chỉ hạ tầng)	Cao hơn (phí quản lý + hạ tầng)
Công sức vận hành	Rất cao (bạn lo mọi thứ)	Thấp (nhà cung cấp lo Control Plane)
Độ khó	Cao	Trung bình
Kiểm soát tầng thấp	Toàn quyền	Hạn chế
Phù hợp khi	Học tập, lab, thử nghiệm, on-prem đặc thù	Production nghiêm túc, đội ngũ nhỏ, cần ổn định

Lời khuyên thực dụng: **Học** thì dùng Minikube trên máy bạn (miễn phí, nhanh). **Production** thì hầu như luôn nên dùng managed (EKS/GKE) trừ khi có lý do đặc biệt để tự quản.

7. Kết bài & nối sang Session 2

Điều quan trọng nhất cần mang theo khỏi buổi hôm nay không phải là tên các thành phần, mà là **tư duy declarative**:

- Bạn **mô tả trạng thái mong muốn**, không ra lệnh từng bước.
- Một **reconcile loop** liên tục kéo thực tế về khớp với mong muốn — đó là gốc của self-healing, scaling và rolling update.
- Mọi giao tiếp đi qua **API Server**; **etcd** giữ sự thật; **Scheduler** xếp chỗ; **Controller Manager** duy trì trạng thái.
- **Pod** là đơn vị nhỏ nhất; **Deployment** quản lý phiên bản; **Service** cho địa chỉ ổn định.
- Trên cloud: **EKS/GKE** là K8s managed, **ECS** là orchestrator riêng của AWS, **Cloud Build** lo phần build image cho pipeline CI/CD.

Câu hỏi tự kiểm tra: Nếu bạn xóa tay một Pod do Deployment quản lý, chuyện gì xảy ra và *tại sao*? (Gợi ý: nghĩ về reconcile loop.)

Session 2 sẽ chuyển từ lý thuyết sang thực hành: cài Minikube/kubectl, viết file YAML đầu tiên, `kubectl apply` một Deployment, và tận mắt thấy reconcile loop hồi sinh một Pod bạn cố tình xóa. Hẹn gặp lại.

Minikube, Pod, Deployment & YAML

90' Dựng cluster local, deploy FE + BE, và hiểu từng dòng YAML.

Mục tiêu buổi học

Sau buổi này bạn sẽ:

- Đọc và viết được file YAML của Kubernetes, hiểu **vai trò của từng khối** thay vì học vẹt.
- Dựng được một cluster Kubernetes chạy ngay trên máy mình bằng **minikube**.
- Deploy được một ứng dụng gồm **Frontend (React/Next)** và **Backend (Go)**.
- Hiểu **tại sao** hầu như không ai deploy Pod trần trong thực tế, mà luôn dùng **Deployment**.
- Biết debug bằng `kubectl logs`, `kubectl describe`, và sinh nhanh manifest với `--dry-run`.

Điều quan trọng nhất của buổi này không phải là nhớ lệnh, mà là hiểu **mô hình tư duy khai báo (declarative)** của Kubernetes. Khi đã nắm được nó, mọi loại tài nguyên khác (Service, Ingress, ConfigMap...) đều đọc theo cùng một khuôn.

1. Cấu trúc file YAML của Kubernetes

Mọi tài nguyên trong Kubernetes — dù là Pod, Deployment hay Service — đều được mô tả bằng một file YAML có **bốn khối gốc** giống nhau:

```
apiVersion: apps/v1      # (1) Nhóm & phiên bản API
kind: Deployment          # (2) Loại tài nguyên
metadata:                 # (3) Thông tin định danh
  name: backend
  labels:
    app: backend
spec:                     # (4) Trạng thái MONG MUỐN
  replicas: 3
  # ...
```

Khối	Vai trò	Vì sao cần
<code>apiVersion</code>	Cho biết dùng nhóm API nào và phiên bản nào (<code>v1</code> , <code>apps/v1</code> , <code>networking.k8s.io/v1</code> ...)	K8s phát triển theo thời gian; versioning giúp cùng một cluster hỗ trợ nhiều schema mà không phá vỡ manifest cũ.
<code>kind</code>	Loại đối tượng: <code>Pod</code> , <code>Deployment</code> , <code>Service</code> ...	Bộ điều khiển (controller) tương ứng sẽ “nhặt” đúng loại này để xử lý.
<code>metadata</code>	Tên, namespace, labels , annotations	<code>name</code> để định danh; <code>labels</code> là “keo dán” để các tài nguyên tìm nhau (Service tìm Pod qua label).
<code>spec</code>	Mô tả trạng thái mong muốn	Đây là trái tim của mô hình declarative.

Vì sao K8s dùng YAML declarative? Bạn **không ra lệnh** “hãy tạo container này, rồi khởi động, rồi...” (imperative). Thay vào đó bạn **khai báo kết quả mong muốn**: “tôi muốn luôn có 3 bản sao của app này đang chạy”. Kubernetes có một vòng lặp điều hòa (*reconciliation loop*) liên tục so sánh **trạng thái thực tế** với **trạng thái mong muốn** và tự động sửa sai. Một Pod chết? Nó tạo lại. Một node hỏng? Nó dời Pod sang node khác. Bạn mô tả *cái đích*, K8s lo *cách đi tới đó*.

Đây cũng là lý do YAML nên được lưu trong Git: file chính là **nguồn sự thật (source of truth)** cho hạ tầng của bạn (GitOps).

2. Pod là gì, và vì sao không nên tạo Pod trần

Pod là đơn vị triển khai nhỏ nhất trong Kubernetes. Một Pod bọc **một hoặc vài container** dùng chung network (cùng địa chỉ IP, cùng cổng) và có thể chia sẻ volume. 90% trường hợp một Pod = một container ứng dụng.

Một Pod trần đơn giản:

```

apiVersion: v1
kind: Pod
metadata:
  name: backend
  labels:
    app: backend
spec:
  containers:
    - name: backend
      image: golang:1.22
      ports:
        - containerPort: 8080

```

Trông thì gọn, nhưng **trong thực tế gần như không ai deploy Pod trần**. Lý do:

- **Không tự hồi sinh đúng nghĩa.** Nếu node chứa Pod bị hỏng, Pod đó biến mất vĩnh viễn — không có gì tạo lại nó ở node khác. (Container bên trong Pod restart được, nhưng bản thân Pod thì không được “di dời”.)
- **Không scale.** Muốn 3 bản sao? Bạn phải copy file 3 lần với 3 cái tên khác nhau. Cực kỳ thủ công.
- **Không rollout / rollback.** Đổi image sang phiên bản mới nghĩa là xóa Pod cũ tạo Pod mới — có downtime, không có cơ chế triển khai từ từ, không quay lui được.

Vì sao vẫn học Pod trần? Vì mọi thứ cấp cao hơn (Deployment, StatefulSet, DaemonSet) cuối cùng cũng **tạo ra Pod**. Hiểu Pod là hiểu viên gạch. Nhưng khi xây nhà thật, bạn dùng cần cẩu (Deployment) chứ không bê từng viên gạch bằng tay.

3. Deployment và ReplicaSet: cần cẩu của Kubernetes

Deployment là tài nguyên bạn sẽ dùng hằng ngày cho ứng dụng *stateless* (FE, BE API). Nó giải quyết đúng 3 vấn đề của Pod trần ở trên.

Cơ chế phân tầng:

```

Deployment —quản lý→ ReplicaSet —quản lý→ Pod, Pod, Pod ...

```

- **ReplicaSet** có đúng một nhiệm vụ: đảm bảo **luôn có đủ số lượng Pod** như khai báo (`replicas: 3`). Một Pod chết → ReplicaSet tạo Pod mới ngay lập tức.

- **Deployment** đứng trên ReplicaSet để quản lý **việc thay đổi phiên bản**.

Vì sao cần thêm một tầng Deployment mà không dùng ReplicaSet trực tiếp? Vì **rolling update**. Khi bạn đổi image (ví dụ v1 → v2), Deployment sẽ tạo một **ReplicaSet mới** cho v2 và **giảm dần** ReplicaSet cũ của v1: tăng 1 Pod v2 → chờ nó khỏe (Ready) → xóa 1 Pod v1 → lặp lại. Nhờ vậy ứng dụng **không bao giờ chết hẳn** trong lúc cập nhật. Và vì ReplicaSet cũ vẫn còn (chỉ bị scale về 0), bạn có thể **rollback** tức thì bằng cách quay lại nó.

Hai trường quan trọng nối Deployment với Pod là `selector.matchLabels` và `template.metadata.labels` — **chúng phải khớp nhau**, nếu không K8s không biết ReplicaSet phải quản Pod nào và sẽ báo lỗi.

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 3
  selector:
    matchLabels:
      app: backend      # ← phải khớp ...
  template:
    metadata:
      labels:
        app: backend    # ← ... với cái này
    spec:
      containers:
        - name: backend
          image: my-backend:1.0
          ports:
            - containerPort: 8080
```

4. minikube làm gì bên dưới?

Để chạy K8s thật trên máy cá nhân, cách dễ nhất là **minikube** — nó dựng một cluster K8s **một node** ngay trên máy bạn.

```
minikube start --driver=docker
```

Bên dưới, lệnh này làm mấy việc:

1. **Tạo một “node” ảo** — thường là một container Docker (hoặc một máy ảo) đóng vai trò vừa là *control plane* vừa là *worker node*.
2. **Cài các thành phần control plane** vào node đó: `kube-apiserver`, `etcd`, `scheduler`, `controller-manager`.
3. **Ghi cấu hình vào kubeconfig** (`~/.kube/config`) và đặt context hiện tại thành `minikube`, để `kubectl` biết phải nói chuyện với API server nào.

Vì sao cần kubeconfig? `kubectl` chỉ là một *client HTTP*. Nó cần biết **địa chỉ API server** và **thông tin xác thực** để gọi tới. `minikube start` tự điền hết vào kubeconfig nên bạn dùng được `kubectl` ngay mà không cấu hình gì thêm.

Kiểm tra:

```
kubectl get nodes
```

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	2m	v1.30.0

`STATUS = Ready` nghĩa là node đã sẵn sàng nhận Pod. Một node duy nhất tên `minikube` đóng cả hai vai — hoàn hảo để học, không dùng cho production.

5. Deploy Frontend (React/Next) và Backend (Go)

Ta sẽ deploy một hệ gồm 2 phần. Để tập trung vào K8s, phần demo dùng image công khai; phần thật dùng image bạn tự build.

5.1. Bắt đầu bằng Pod (để thấy vấn đề)

`pod-frontend.yaml` (dùng `nginx` làm FE demo):

```
apiVersion: v1
kind: Pod
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  containers:
    - name: frontend
      image: nginx:1.27
      ports:
        - containerPort: 80
```

Apply và kiểm tra:

```
kubectl apply -f pod-frontend.yaml
kubectl get pods
kubectl describe pod frontend      # xem sự kiện: pull image, schedule, start
kubectl logs frontend              # xem log stdout của container
```

Vì sao `describe` quan trọng khi debug? Phần **Events** ở cuối output kể lại “tiểu sử” của Pod: đã được scheduler gán vào node chưa, kéo image thành công không, container khởi động lỗi gì. 80% lỗi (`ImagePullBackOff` , `CrashLoopBackOff`) được chẩn đoán ngay tại đây trước khi cần đọc log.

5.2. Chuyển sang Deployment (cách làm thật)

`deployment-backend.yaml` (BE Go, ở đây dùng image bạn build; demo có thể thay bằng image bất kỳ):

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: my-backend:1.0          # image Go tự build
          ports:
            - containerPort: 8080

```

```

kubectl apply -f deployment-backend.yaml
kubectl get deploy,rs,pods          # thấy cả 3 tầng cùng lúc

```

5.3. Scale và rollout

```

# Scale lên 5 bản sao
kubectl scale deployment/backend --replicas=5

# Cập nhật image sang phiên bản mới (kích hoạt rolling update)
kubectl set image deployment/backend backend=my-backend:2.0

# Theo dõi tiến trình rollout
kubectl rollout status deployment/backend

# Lỡ bản mới lỗi? Quay lui ngay lập tức
kubectl rollout undo deployment/backend

```

Vì sao `rollout undo` nhanh đến vậy? Vì ReplicaSet cũ **không bị xóa**, chỉ bị scale về 0. Rollback chỉ là scale nó trở lại và scale ReplicaSet mới về 0 — không cần build hay kéo image lại.

6. Mẹo: dùng AI và `--dry-run` để sinh nhanh manifest

Viết YAML bằng tay dễ sai thật lè. Mẹo chuẩn của dân K8s là để `kubectl` **tự sinh khung** rồi mới sửa:

```
kubectl create deployment backend \
  --image=my-backend:1.0 \
  --replicas=2 \
  --dry-run=client -o yaml > deployment-backend.yaml
```

- `--dry-run=client` : **không thực sự tạo** gì trên cluster, chỉ dựng object trong bộ nhớ.
- `-o yaml` : in ra YAML.

Vì sao cách này tốt hơn chép mẫu trên mạng? Vì YAML sinh ra luôn khớp với **phiên bản K8s bạn đang chạy** (đúng `apiVersion`, đúng schema). Sau đó bạn dán vào AI/editor và chỉ cần chỉnh những trường thật sự cần (probes, resources, env). Ít lỗi hơn, nhanh hơn, và bạn vẫn hiểu mình đang sửa gì.

Tương tự có thể tạo nhanh khung cho Service, ConfigMap... bằng `kubectl create ... --dry-run=client -o yaml`.

Tổng kết

- YAML của K8s luôn gồm 4 khối: `apiVersion`, `kind`, `metadata`, `spec`. `spec` mô tả **trạng thái mong muốn**, K8s tự điều hòa để đạt được.
- Pod là viên gạch nhỏ nhất nhưng **không tự hồi sinh / scale / rollout** → thực tế dùng **Deployment**.
- Deployment quản ReplicaSet, ReplicaSet giữ đủ số Pod; nhờ hai tầng này mà có **rolling update** và **rollback** tức thì.
- `minikube start` dựng cluster 1 node và điền kubeconfig để `kubectl` dùng ngay.
- Debug bằng `kubectl describe` (xem Events) và `kubectl logs`.
- Sinh manifest nhanh bằng `--dry-run=client -o yaml` để tránh lỗi cú pháp.

Buổi tới ta sẽ cho FE và BE **nói chuyện với nhau** qua Service và mở ra ngoài — nhưng nền tảng “trạng thái mong muốn” của hôm nay là thứ bạn sẽ dùng mãi.

Networking, Multi-env & RBAC

90' Kết nối FE↔BE bằng Service, tách môi trường bằng Namespace, kiểm soát quyền bằng RBAC.

Mục tiêu buổi học

Sau buổi 2 bạn đã biết deploy Pod và Deployment. Nhưng một ứng dụng thực tế không chỉ có một khối chạy độc lập: frontend phải gọi được backend, backend phải gọi được database, và tất cả phải sống sót qua các lần Pod chết đi rồi sinh lại. Buổi này trả lời ba câu hỏi lớn:

1. Làm sao các thành phần **tìm và gọi nhau ổn định** khi IP luôn thay đổi? (Service + DNS)
2. Làm sao **tách môi trường** dev/staging/prod mà không dựng nhiều cluster? (Namespace)
3. Làm sao **kiểm soát ai được làm gì** trong cluster? (RBAC)

Trọng tâm buổi này là RBAC — phần dễ bị bỏ qua nhất nhưng lại quyết định độ an toàn của cả hệ thống.

1. Vì sao Pod IP không ổn định

Mỗi Pod khi khởi tạo được cấp một IP riêng trong mạng nội bộ của cluster. Nghe thì tiện, nhưng IP này **gắn với vòng đời của Pod**, không gắn với ứng dụng.

Khi nào Pod chết và sinh lại?

- Node chứa Pod bị lỗi hoặc restart.
- Bạn `kubectl rollout` một phiên bản mới → Pod cũ bị xóa, Pod mới lên.
- HPA/scaling tăng giảm số replica.
- Liveness probe fail → kubelet kill và tạo lại Pod.

Mỗi lần như vậy, Pod mới **gần như chắc chắn có IP khác**. Nếu frontend hard-code IP của backend, chỉ cần một lần backend rollout là frontend gọi vào hư không.

Vì sao: Pod là “cattle, not pets” — bị coi như vật thể dùng-rồi-thay. Ta không được phụ thuộc vào danh tính (IP) của một Pod cụ thể, mà cần một lớp trừu tượng ổn định đứng phía trước nhóm Pod. Đó chính là Service.

2. Các loại Service

Service là một object có **IP ảo (ClusterIP)** và **tên DNS cố định**, đóng vai trò cân bằng tải tới nhóm Pod được chọn qua `selector`. Pod chết sinh lại tùy ý, nhưng Service không đổi.

Loại	Phạm vi truy cập	Dùng khi nào
ClusterIP (mặc định)	Chỉ trong cluster	Giao tiếp nội bộ: FE→BE, BE→cache. Không muốn lộ ra ngoài.
NodePort	Mở một port (30000–32767) trên mọi Node	Test nhanh, môi trường dev/lab, không có LoadBalancer.
LoadBalancer	IP public qua load balancer của cloud	Production trên cloud (AWS/GCP/Azure).
Ingress (không phải Service)	HTTP/HTTPS routing theo host/path	Nhiều dịch vụ chung một endpoint, cần TLS, định tuyến theo domain.

Vì sao chọn ClusterIP cho backend: backend không cần và không nên truy cập trực tiếp từ ngoài. Mở ra ngoài là mở thêm bề mặt tấn công. Cứ để nó ClusterIP, chỉ frontend trong cluster gọi được.

NodePort mở cùng một port trên mọi Node và forward vào Service. Tiện cho lab nhưng ở production ít dùng vì port cao khó nhớ, không có TLS, và phơi bày Node ra ngoài. Trên cloud thật, ta dùng LoadBalancer hoặc (phổ biến hơn) một Ingress Controller đứng trước nhiều Service.

3. Service discovery bằng DNS nội bộ

Điểm hay nhất của Service là bạn **không cần biết IP của nó**. Kubernetes chạy một DNS server nội bộ tên **CoreDNS**. Mỗi Service được cấp một bản ghi DNS theo mẫu:

```
<tên-service>.<namespace>.svc.cluster.local
```

Nên frontend chỉ cần gọi:

```
http://backend-svc:8080
```

Cơ chế hoạt động:

1. Khi tạo Service `backend-svc` trong namespace `default`, CoreDNS tự tạo bản ghi A trả về `backend-svc.default.svc.cluster.local` về ClusterIP của Service.
2. Mỗi Pod được kubelet cấu hình `/etc/resolv.conf` trả về CoreDNS, kèm `search default.svc.cluster.local ...`.
3. Vì cùng namespace nằm trong `search domain`, gõ ngắn gọn `backend-svc` là đủ; khác namespace thì cần `backend-svc.tên-namespace`.
4. Traffic tới ClusterIP được kube-proxy (qua iptables/IPVS) phân phối tới các Pod đang khỏe mạnh.

Vì sao dùng tên thay vì IP: tên DNS là hợp đồng ổn định giữa các dịch vụ. IP đằng sau có thể đổi hàng chục lần mỗi ngày, nhưng `backend-svc:8080` thì bất biến. Đây là nền tảng của kiến trúc microservice trên K8s.

4. Namespace — tách môi trường

Namespace là cách chia một cluster thành nhiều “phòng” logic. Object trong namespace này không nhìn thấy object trong namespace khác (trừ khi gọi qua DNS đầy đủ).

Vì sao tách dev/staging bằng namespace:

- **Cô lập tên:** cả hai môi trường đều có Deployment tên `web`, Service tên `backend-svc` mà không đụng nhau.
- **Resource Quota:** giới hạn CPU/RAM cho `dev` để một môi trường không nuốt hết tài nguyên của môi trường khác.
- **Phân quyền (RBAC):** gán quyền theo namespace — `dev-user` chỉ được thao tác trong `dev`, không đụng được `staging`.

So với việc tách bằng **cluster riêng cho mỗi môi trường**:

Tiêu chí	Namespace	Cluster riêng
Chi phí hạ tầng	Thấp (chung control plane)	Cao (mỗi cluster một bộ)
Mức cô lập	Logic (mềm)	Vật lý (cứng)
Vận hành	Đơn giản, một context	Phức tạp, nhiều context
Phù hợp	dev/staging, team nhỏ	prod tách biệt hoàn toàn, tuân thủ bảo mật cao

Vì sao thường dùng namespace cho dev/staging: rẻ và nhanh, đủ cô lập cho các môi trường không phải production. Production nhạy cảm thì nhiều tổ chức vẫn tách cluster riêng để cô lập vật lý sự cố và bán kính ảnh hưởng khi bị xâm nhập.

5. RBAC — trọng tâm

RBAC (Role-Based Access Control) là cơ chế trả lời câu hỏi: **“Chủ thể X có được thực hiện hành động Y trên tài nguyên Z không?”**

5.1 Vì sao cần RBAC

Mặc định, nếu ai cũng dùng chung một kubeconfig admin thì ai cũng có thể xóa Deployment production, đọc mọi Secret, tạo Pod chạy tùy ý. Đó là thảm họa chờ xảy ra.

Vì sao least privilege: mỗi người/dịch vụ chỉ nên có đúng quyền tối thiểu cần cho công việc. Một dev CI chỉ cần deploy vào `dev` thì không được có quyền đọc Secret production. Nếu token của họ bị lộ, thiệt hại bị giới hạn trong phạm vi hẹp thay vì toàn cluster.

5.2 Các thành phần

RBAC ghép hai nửa: **định nghĩa quyền** (Role/ClusterRole) và **gán quyền cho chủ thể** (RoleBinding/ClusterRoleBinding).

Object	Phạm vi	Ý nghĩa
Role	Trong một namespace	Tập hợp quyền, chỉ có hiệu lực trong namespace đó
ClusterRole	Toàn cluster	Quyền cho tài nguyên cluster-scoped (node, PV) hoặc dùng lại ở nhiều namespace
RoleBinding	Trong một namespace	Gán một Role (hoặc ClusterRole) cho chủ thể, giới hạn trong namespace
ClusterRoleBinding	Toàn cluster	Gán ClusterRole cho chủ thể trên toàn cluster

Về chủ thể (subject):

- **User**: con người, quản lý bên ngoài K8s (certificate, OIDC...). K8s không có object "User".
- **ServiceAccount**: danh tính cho tiến trình/Pod hoặc CI/CD chạy trong cluster. Đây là object thật trong K8s, dùng cho tự động hóa.

Vì sao tách Role và Binding: Role định nghĩa "được làm gì", Binding định nghĩa "ai được nhận Role đó". Tách ra để tái sử dụng: một Role `viewer` có thể gán cho nhiều người khác nhau mà không viết lại quyền.

5.3 Verbs và Resources

Một `rule` trong Role gồm ba phần: `apiGroups`, `resources` và `verbs`.

- **verbs** — hành động: `get`, `list`, `watch` (đọc), `create`, `update`, `patch`, `delete` (ghi).
- **resources** — đối tượng: `pods`, `deployments`, `services`, `secrets`, `configmaps` ...
- **apiGroups** — nhóm API: `""` (core: pod, service, configmap), `apps` (Deployment, ReplicaSet)...

Ví dụ một Role chỉ cho đọc Pod:

```
rules:
- apiGroups: ["" ]
  resources: ["pods"]
  verbs: ["get", "list", "watch"]
```

5.4 Kiểm chứng bằng kubectl auth can-i

Đừng đoán RBAC có đúng không — hãy kiểm tra bằng lệnh:

```
# Hỏi thay cho một ServiceAccount cụ thể
kubectl auth can-i list pods \
  --namespace dev \
  --as=system:serviceaccount:dev:dev-user

# Trả về "yes" hoặc "no"
kubectl auth can-i create deployments \
  --namespace staging \
  --as=system:serviceaccount:dev:dev-user
```

Vì sao dùng `auth can-i` : nó mô phỏng chính xác quyết định của API server mà không cần thực sự thực hiện hành động nguy hiểm. Đây là công cụ vàng để chứng minh least privilege đã cấu hình đúng trước khi lên production.

Tổng kết

- Pod IP không ổn định → dùng **Service** làm điểm truy cập bất biến.
- **ClusterIP** cho nội bộ, **NodePort** cho lab, **LoadBalancer/Ingress** cho production.
- **CoreDNS** cho phép gọi nhau bằng tên `service.namespace` thay vì IP.
- **Namespace** tách môi trường rẻ và nhanh; cluster riêng khi cần cô lập cứng.
- **RBAC** thực thi least privilege: Role/ClusterRole định nghĩa quyền, RoleBinding/ClusterRoleBinding gán cho User/ServiceAccount, kiểm chứng bằng `kubectl auth can-i`.

Phần Lab sẽ thực hành lần lượt cả ba: Service + DNS, Namespace dev/staging, và RBAC chặn deploy chéo môi trường.

AI Deployment + Managed K8s (EKS/ECS)

90' Deployment chuẩn production với probes, và mở rộng lên EKS/ECS/Cloud Build.

Session 4 — AI Deployment + Managed K8s (EKS/ECS)

Ở các session trước, chúng ta đã chạy được Pod và biết cách phoir ứng dụng qua Service. Nhưng một Pod đơn lẻ chưa đủ để gọi là “production”. Trong bài này, ta trả lời câu hỏi cốt lõi: **làm sao để ứng dụng luôn sống, tự phục hồi, không rớt traffic khi cập nhật, và chạy được trên hạ tầng cloud thật (EKS/ECS)?**

Trọng tâm không phải là “gõ lệnh gì”, mà là **vì sao** Kubernetes thiết kế các cơ chế đó, và điều gì xảy ra nếu bạn cấu hình sai.

1. Deployment chuẩn production: vì sao cần nhiều replicas?

Một Pod trần (bare Pod) có một vấn đề chí mạng: nếu node chứa nó chết, hoặc bạn lỡ tay xóa nó, thì **không có ai đứng ra tạo lại**. Pod biến mất là biến mất luôn.

Deployment là một lớp quản lý cao hơn. Bạn khai báo “tôi muốn luôn có N bản sao (replica) của ứng dụng này”, và Kubernetes qua ReplicaSet sẽ liên tục so sánh **trạng thái mong muốn (desired state)** với **trạng thái thực tế (actual state)**, rồi tự điều chỉnh cho khớp.

Vì sao: Kubernetes vận hành theo mô hình *declarative* (khai báo) chứ không *imperative* (ra lệnh). Bạn mô tả kết quả mong muốn, control loop tự làm phần còn lại. Đây là lý do hệ thống có khả năng “tự chữa lành”.

Việc chạy **nhiều replicas** (thường tối thiểu 3 cho production) mang lại ba lợi ích:

Lợi ích	Giải thích
High Availability (HA)	Một replica chết, các replica còn lại vẫn phục vụ. Người dùng không bị gián đoạn.
Chịu tải (load)	Traffic được Service chia đều ra nhiều Pod, mỗi Pod gánh một phần.
Rolling update không downtime	Khi deploy phiên bản mới, K8s thay từng Pod một, luôn giữ đủ Pod sống để phục vụ.

Về **rolling update**: khi bạn đổi image trong Deployment, K8s không giết hết Pod cũ rồi mới tạo Pod mới (như vậy sẽ có khoảng chết). Thay vào đó nó tuân theo chiến lược `RollingUpdate` với hai tham số:

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxUnavailable: 1 # tối đa 1 Pod được phép "thiếu" trong lúc update
    maxSurge: 1       # cho phép tạo thêm 1 Pod vượt số replicas trong lúc update
```

Vì sao: `maxUnavailable` và `maxSurge` là cách bạn đánh đổi giữa *tốc độ deploy* và *khả năng phục vụ*. Đặt `maxUnavailable: 0` cùng `maxSurge: 1` là an toàn nhất (luôn đủ Pod), nhưng deploy chậm hơn vì phải chờ Pod mới sẵn sàng trước khi gỡ Pod cũ. Chính ở đây, `readinessProbe` (mục sau) mới là thứ quyết định “Pod mới đã sẵn sàng chưa”.

Với ba replicas, nếu chẳng may một Pod bị `delete`, Deployment lập tức tạo lại Pod thứ ba — đây chính là hành vi **self-healing** mà bài lab sẽ kiểm chứng.

2. livenessProbe vs readinessProbe — điểm khác biệt sống còn

Đây là phần bị hiểu sai nhiều nhất. Hai probe nghe na ná nhau nhưng **giải quyết hai vấn đề hoàn toàn khác nhau** và hậu quả khi nhầm lẫn rất khác nhau.

livenessProbe — “Container này còn sống không?”

Kubernetes định kỳ kiểm tra liveness. Nếu probe **thất bại liên tục** vượt ngưỡng, K8s kết luận container đã “chết hoặc treo” và **khởi động lại (restart) chính container đó**.

Dùng cho các tình huống: deadlock, vòng lặp treo, memory leak khiến app không phản hồi nữa. Restart là cách “đá cho tỉnh”.

readinessProbe — “Container này đã sẵn sàng nhận traffic chưa?”

Nếu readiness thất bại, K8s **KHÔNG restart** container. Thay vào đó, nó **gỡ Pod khỏi danh sách endpoints của Service**, tức là ngừng gửi traffic tới Pod đó cho đến khi nó ready trở lại.

Dùng cho: app đang khởi động chưa nạp xong config/cache, hoặc tạm thời quá tải, hoặc đang mất kết nối tới database phía sau.

Hậu quả khi cấu hình SAI — đây mới là bài học

Sai lầm	Hậu quả
Dùng liveness cho việc lẽ ra là readiness (ví dụ: liveness fail khi DB tạm chậm)	K8s restart liên tục container. DB chậm không được chữa bằng restart → CrashLoopBackOff , làm tình hình tệ hơn.
<code>initialDelaySeconds</code> của liveness quá ngắn so với thời gian khởi động app	App chưa kịp boot đã bị coi là “chết” → restart vô hạn, không bao giờ chạy được.
Không có readiness , chỉ có liveness	Trong lúc rolling update, Pod mới nhận traffic ngay khi vừa start (dù chưa nạp xong) → user gặp lỗi 500.
readiness quá nhạy (<code>failureThreshold = 1</code>)	Một cú ngھn nhẹ cũng khiến Pod bị gỡ khỏi Service → mất năng lực phục vụ không cần thiết.

Vì sao: Nguyên tắc vàng — **liveness quyết định SỐNG/CHẾT (restart), readiness quyết định NHẬN TRAFFIC hay không (endpoints)**. Nhầm hai cái này là nguyên nhân số một của các sự cố “app cứ restart mãi không lên” trong thực tế.

startupProbe — nhắc nhở

Với app khởi động **rất chậm** (JVM lớn, migration DB lúc boot...), việc dùng `initialDelaySeconds` lớn cho liveness là vụng về. `startupProbe` sinh ra để giải quyết: trong lúc startupProbe chưa pass, liveness và readiness **bị hoãn**. Khi startupProbe pass lần đầu, liveness mới bắt đầu hoạt động. Nhờ đó app chậm không bị liveness “giết oan” lúc khởi động, mà sau khi lên rồi vẫn được giám sát chặt.

3. resources: requests và limits — vì sao bắt buộc phải khai báo

Mỗi container nên khai báo tài nguyên:

```
resources:
  requests:
    cpu: "100m"      # 0.1 core — mức TỐI THIỂU đảm bảo
    memory: "128Mi"
  limits:
    cpu: "500m"      # trần CPU
    memory: "256Mi"  # trần RAM — vượt là bị giết
```

- **requests** là lượng tài nguyên K8s **cam kết dành riêng** cho container. Scheduler dùng con số này để **quyết định đặt Pod vào node nào** (node phải còn đủ request chưa dùng).
- **limits** là **trần cứng**. CPU vượt limit thì bị *throttle* (bóp lại, chậm đi). RAM vượt limit thì container bị **OOMKilled** (Out Of Memory) — kernel giết tiến trình.

Vì sao cần requests: Không khai request thì scheduler “mù” về nhu cầu thật, có thể nhồi quá nhiều Pod lên một node, gây tranh chấp và sập cả node. Request là cơ sở để scheduling công bằng và ổn định.

Vì sao cần limits: Không có limit RAM thì một Pod rò rỉ bộ nhớ có thể ăn hết RAM của node, kéo theo các Pod hàng xóm chết chum (“noisy neighbor”). Limit là tường lửa cô lập sự cố.

Lưu ý về OOMKill: nếu bạn thấy Pod restart với lý do `OOMKilled`, gần như chắc chắn `memory.limits` đặt quá thấp so với nhu cầu thực — cần đo lại và nâng lên, chứ không phải lỗi “ngẫu nhiên”.

4. Dùng AI để sinh manifest — nhanh, nhưng phải review

AI (như Claude) rất giỏi việc sinh nhanh một Deployment/Service YAML từ mô tả bằng lời. Đây là cách viết prompt tốt:

```
Viết một Kubernetes Deployment cho app Go tên "go-api":  
- image: myrepo/go-api:1.0.0, cổng 8080  
- 3 replicas  
- livenessProbe httpGet /healthz, readinessProbe httpGet /ready  
- resources: requests 100m/128Mi, limits 500m/256Mi  
- kèm một Service kiểu ClusterIP cổng 80 → 8080  
Trả về YAML thuần, có comment giải thích từng probe.
```

Prompt tốt = **nêu rõ ràng buộc cụ thể** (tên, image, cổng, số replica, ngưỡng probe, tài nguyên).
Prompt mơ hồ ("viết deployment cho tôi") sẽ nhận về giá trị mặc định chung chung, thường không hợp production.

Vì sao phải review, không tin mù quáng: AI có thể (1) đặt `initialDelaySeconds` không phù hợp với app của bạn, (2) bịa tên image/registry, (3) quên `readinessProbe`, (4) dùng `latest` tag (chống chỉ định trong production vì không tái lập được), hoặc (5) đặt limits phi thực tế. Manifest là hạ tầng chạy thật — một dòng sai có thể gây CrashLoopBackOff hoặc lỗi hỏng bảo mật. **Luôn đọc lại từng trường, đối chiếu với app thật, rồi mới apply.**

Checklist review nhanh: image tag cố định (không `latest`)? probe trả đúng endpoint có thật trong code? requests/limits hợp lý? số replica đúng? có Service khớp `selector` với `labels` của Pod không?

5. Managed Kubernetes: từ Minikube lên cloud thật

Minikube tuyệt vời để học, nhưng nó là cụm **một node chạy trên máy bạn** — không có HA thật, tắt máy là mất. Production cần managed platform.

EKS (Amazon Elastic Kubernetes Service)

Với EKS, **AWS quản lý control plane** (API server, etcd, scheduler) — bạn không phải tự vá lỗi, sao lưu etcd, hay lo control plane sập. Bạn chỉ quản các worker node (hoặc dùng Fargate để khỏi quản luôn node).

Tạo cụm nhanh nhất bằng `eksctl`:

```
eksctl create cluster \
  --name go-demo \
  --region ap-southeast-1 \
  --nodes 3 \
  --node-type t3.small
```

Điểm mấu chốt: **manifest của bạn KHÔNG đổi**. Cùng file Deployment/Service chạy trên Minikube sẽ chạy y hệt trên EKS (chỉ khác Service `LoadBalancer` sẽ tạo ra một ELB thật của AWS thay vì phải `minikube tunnel`).

	Minikube	EKS
Số node	1 (máy bạn)	Nhiều, đa vùng khả dụng
Control plane	Bạn chịu	AWS quản, có SLA
LoadBalancer	Giả lập / tunnel	ELB thật
Mục đích	Học, dev	Production

ECS (Elastic Container Service) — orchestration KHÔNG dùng K8s

ECS là bộ điều phối container **riêng của AWS**, không phải Kubernetes. Hai khái niệm cốt lõi:

- **Task Definition**: bản mô tả "chạy container nào, image gì, CPU/RAM bao nhiêu, cổng nào" — vai trò hao hao Pod spec.
- **Service**: đảm bảo luôn có N task chạy và gắn với load balancer — vai trò hao hao Deployment + Service của K8s.

Vì sao/khi nào chọn ECS thay EKS: Chọn ECS khi bạn đã "all-in AWS", muốn vận hành **đơn giản nhất có thể**, không cần hệ sinh thái add-on phong phú của K8s, và team không muốn học đường cong dốc của Kubernetes. Chọn EKS khi bạn cần chuẩn mở, đa cloud, hoặc đã có sẵn manifest/kỹ năng K8s. ECS đổi lấy sự đơn giản bằng việc **khóa vào AWS (vendor lock-in)**.

Cloud Build / GKE — nối sang CI/CD

Đích cuối là **tự động hóa**: lập trình viên push code, phần còn lại tự chạy. Trên Google Cloud, luồng điển hình:

```
git push → Cloud Build (theo trigger từ commit)
          → build Docker image
          → push image lên Artifact Registry
          → kubectl apply / deploy lên GKE
```

Đây chính là cầu nối sang chủ đề **CI/CD** của session tiếp theo: image được build từ đúng commit, gắn tag bằng SHA của commit (tái lập được), và deploy tự động. Con người không còn `docker build` bằng tay nữa.

Vì sao đây là bước trưởng thành: khi build/deploy tự động và tái lập được, bạn loại bỏ lỗi “chạy được trên máy tôi”, và mỗi lần deploy đều có dấu vết truy được về commit. Đó là nền tảng của vận hành production đáng tin cậy.

Tổng kết

- **Deployment + nhiều replicas** cho HA, chịu tải và rolling update không downtime.
- **liveness** quyết định restart (sống/chết); **readiness** quyết định nhận traffic hay không. Nhầm lẫn = CrashLoopBackOff hoặc rớt traffic.
- **requests/limits** giữ cho scheduling ổn định và cô lập sự cố; sai limit RAM = OOMKilled.
- Dùng **AI để sinh manifest** cho nhanh, nhưng **luôn review** trước khi apply.
- **EKS/ECS/GKE** đưa bạn từ Minikube lên hạ tầng thật; cùng manifest, control plane do cloud quản.

Trong bài lab, bạn sẽ tự tay xây một app Go với `/healthz` và `/ready`, đóng gói Docker, và deploy bằng Deployment 3 replicas có đủ probe — rồi kiểm chứng self-healing.

CI/CD: GitHooks → Jenkins → Deploy

90' Vì sao cần CI/CD, GitHooks, dựng Jenkins bằng Docker và pipeline Build–Test–Deploy.

Mục tiêu buổi học

Sau buổi này bạn sẽ hiểu **tại sao** một quy trình CI/CD lại quan trọng hơn nhiều so với việc deploy tay, biết dùng **GitHooks** đúng chỗ, dựng được **Jenkins bằng Docker** và viết một **pipeline Build → Test → Deploy** cơ bản lên Kubernetes.

Trọng tâm không phải là “gõ đúng lệnh”, mà là hiểu vì sao mỗi mảnh ghép tồn tại. Khi hiểu lý do, bạn tự thiết kế được pipeline cho dự án của mình thay vì sao chép mù quáng.

1. Vì sao cần CI/CD?

Hãy tưởng tượng quy trình deploy thủ công điển hình: lập trình viên build code trên máy mình, copy file lên server bằng SCP, SSH vào chạy vài lệnh, rồi restart service. Cách này có ba vấn đề chí mạng:

- **Dễ lỗi (error-prone):** con người quên bước, gõ nhầm lệnh, deploy nhầm nhánh. Mỗi lần deploy là một lần “cầu nguyện”.
- **Chậm:** mỗi lần release tốn hàng chục phút thao tác tay, không ai dám deploy nhiều lần trong ngày.
- **Không lặp lại được (not reproducible):** “máy tôi chạy được mà” — vì môi trường build của mỗi người mỗi khác. Không có bản ghi nào nói chính xác artifact này được build ra sao.

Vì sao tự động hóa: máy móc làm đúng một quy trình hàng nghìn lần không mệt, không quên. CI/CD biến việc release từ một *sự kiện đáng sợ* thành một *thao tác thường ngày*.

CI và CD khác nhau thế nào?

Hai khái niệm hay bị gộp làm một nhưng giải quyết vấn đề khác nhau:

Khái niệm	Viết tắt	Trọng tâm	Câu hỏi trả lời
Continuous Integration	CI	Tích hợp code liên tục, tự động build + test mỗi khi có commit	"Code mới có làm hỏng gì không?"
Continuous Delivery	CD	Luôn giữ code ở trạng thái <i>sẵn sàng release</i> ; deploy cần một cú bấm nút	"Ta có thể release bất cứ lúc nào không?"
Continuous Deployment	CD	Tự động deploy lên production khi qua hết test, không cần người bấm	"Có tự động lên production không?"

Vì sao tách CI và CD: CI bảo vệ chất lượng code (bắt lỗi sớm, khi sửa còn rẻ). CD bảo vệ tốc độ giao hàng. Một dự án có thể làm CI trước, chín muồi rồi mới tiến tới CD.

2. GitHooks — tự động hóa ngay tại Git

Git hook là các script Git tự chạy tại những thời điểm nhất định trong vòng đời commit/push. Chúng nằm trong thư mục `.git/hooks/`. Đây là *tuyến phòng thủ đầu tiên*, chạy ngay trên máy dev trước cả khi code rời khỏi laptop.

Client-side hooks (chạy trên máy dev)

Hook	Kích hoạt khi	Dùng để làm gì và <i>tại sao</i>
<code>pre-commit</code>	Trước khi commit được tạo	Chạy linter, format, kiểm tra secret. <i>Vì sao</i> : chặn code bẩn ngay từ đầu, rẻ hơn sửa sau.
<code>commit-msg</code>	Sau khi nhập message, trước khi lưu commit	Ép định dạng message (vd Conventional Commits). <i>Vì sao</i> : lịch sử git đọc được, tự sinh changelog.
<code>post-commit</code>	Sau khi commit hoàn tất	Thông báo, ghi log. <i>Vì sao</i> : commit đã xong nên chỉ dùng cho việc phụ, không chặn được commit.
<code>pre-push</code>	Trước khi đẩy lên remote	Chạy test nhanh, chặn push lên nhánh bảo vệ. <i>Vì sao</i> : rào chắn cuối trước khi code ảnh hưởng người khác.

Server-side hook

Hook	Kích hoạt khi	Dùng để làm gì và <i>tại sao</i>
<code>post-receive</code>	Trên server Git, sau khi nhận push xong	Trigger deploy, gọi Jenkins, gửi thông báo. Vì <i>sao</i> : chạy tập trung nên không thể bị dev bỏ qua.

Vì sao phân biệt client-side vs server-side: client-side hook **có thể bị bỏ qua** (dev xóa hook, hoặc `git commit --no-verify`). Vì vậy đừng bao giờ coi chúng là bảo đảm cuối cùng. Kiểm tra *bắt buộc* phải đặt ở phía server (server-side hook hoặc trên CI), nơi dev không can thiệp được.

3. Luồng CI/CD tổng thể

Ghép các mảnh lại, một pipeline điển hình chạy như sau:

```
commit → (client hook: pre-commit/pre-push kiểm tra cục bộ)
        → push lên Git server
        → webhook / post-receive hook báo cho Jenkins
        → Jenkins: Checkout code
        → Build
        → Unit Test
        → Build Docker image + push lên registry
        → Deploy lên Kubernetes (kubectl apply)
```

Vì sao dùng webhook thay vì để Jenkins hỏi liên tục (polling): webhook là *push* — commit tới đâu Jenkins biết ngay đó, phản hồi tức thì và không lãng phí tài nguyên hỏi Git mỗi phút một lần.

Mỗi bước là một “cổng chất lượng”. Nếu unit test fail, pipeline dừng và **không** có image nào được build, không gì được deploy. Đây chính là giá trị: lỗi bị chặn *trước khi* chạm tới người dùng.

4. Vì sao chạy Jenkins trong Docker?

Jenkins có thể cài trực tiếp lên máy (WAR file, gói apt...), nhưng chạy trong Docker là lựa chọn hiện đại vì:

- **Môi trường sạch:** container khởi tạo từ image chuẩn, không dính “rác” cấu hình của máy chủ. Ai dựng cũng ra kết quả giống nhau.
- **Dễ dựng lại (reproducible):** cần Jenkins mới? Chỉ một lệnh `docker run`. Máy hỏng? Dựng lại trong vài phút, dữ liệu nằm ở volume.
- **Versioned:** ghim đúng phiên bản qua tag image (vd `jenkins/jenkins:lts-jdk17`). Nâng cấp hay rollback đều rõ ràng, không mơ hồ.
- **Cô lập:** Jenkins không đụng chạm phần mềm khác trên host.

Vì sao vẫn cần volume dù chạy trong Docker: container vốn *ephemeral* (xóa là mất sạch). Dữ liệu Jenkins (job, plugin, cấu hình) phải nằm ở volume bền vững, nếu không xóa container là mất hết công cấu hình.

5. Jenkinsfile — Pipeline as Code

Jenkins cổ điển cấu hình job qua giao diện web (**Freestyle job**): bấm chuột điền form. Cách hiện đại là viết pipeline vào file `Jenkinsfile` đặt ngay trong repo — gọi là **Pipeline as Code**.

Ví dụ một declarative pipeline tối giản:

```
pipeline {
  agent any
  stages {
    stage('Build') {
      steps {
        echo 'Đang build ứng dụng ... '
      }
    }
    stage('Test') {
      steps {
        echo 'Đang chạy unit test ... '
      }
    }
  }
}
```

Trong đó:

- `agent any` — chạy trên bất kỳ node/executor nào Jenkins có.
- `stages` — chuỗi giai đoạn, hiển thị trực quan trên UI.

- `stage` / `steps` — mỗi giai đoạn gồm các lệnh cụ thể.

Vì sao Pipeline-as-code tốt hơn cấu hình UI?

Tiêu chí	Freestyle (UI)	Jenkinsfile (code)
Version control	Không (cấu hình nằm trong Jenkins)	Có — nằm trong Git, review được qua Pull Request
Lịch sử thay đổi	Khó truy vết	<code>git log</code> rõ từng dòng, ai sửa gì
Nhân bản dự án	Chép tay từng ô	Copy file, sửa vài dòng
Khôi phục khi Jenkins hỏng	Mất sạch nếu không backup	Còn nguyên trong repo
Code review	Không	Có, như mọi code khác

Vì sao đây là bước ngoặt: khi pipeline là code nằm cùng ứng dụng, nó tiến hóa *cùng* code. Đổi cách build là một commit, được review, có lịch sử — thay vì một thay đổi “vô hình” ai đó bấm trên UI mà không ai biết.

6. Bảo mật cơ bản

CI/CD chạm tới registry, cluster, server production — nếu lộ credential, hậu quả rất nặng. Vài nguyên tắc tối thiểu:

- **Không hardcode secret** (mật khẩu, token, kubeconfig) vào Jenkinsfile hay commit lên Git. Một khi vào lịch sử Git, secret coi như đã lộ vĩnh viễn.
- Dùng **Jenkins Credentials Store**: lưu secret tại đó, trong pipeline gọi qua `credentials()` hoặc block `withCredentials`.

```
environment {
    REGISTRY_CRED = credentials('docker-registry-cred')
}
```

- **Nguyên tắc quyền tối thiểu**: token/service account chỉ có đúng quyền cần thiết. Token deploy không cần quyền xóa cả cluster.

- **Xoay vòng (rotate) credential** định kỳ và khi nghi ngờ lộ.

Vì sao coi trọng ngay từ đầu: pipeline tự động nghĩa là secret được dùng *tự động, liên tục*. Một secret lộ trong CI/CD có thể bị lạm dụng hàng loạt trước khi ai kịp nhận ra. An toàn phải là mặc định, không phải bổ sung sau.

Tổng kết

- Deploy tay chậm, dễ lỗi, không lặp lại được — CI/CD giải quyết cả ba.
- **CI** giữ chất lượng code; **CD** giữ tốc độ giao hàng.
- **GitHooks** là tuyến phòng thủ đầu tiên; client-side có thể bị bỏ qua nên kiểm tra bắt buộc phải ở server.
- Chạy **Jenkins trong Docker** cho môi trường sạch, dễ dựng lại, có version — nhớ gắn volume để giữ dữ liệu.
- **Jenkinsfile (Pipeline as Code)** hơn hẳn Freestyle vì được version, review và khôi phục được.
- **Không bao giờ hardcode secret**; dùng credentials store và quyền tối thiểu.

Ở phần Lab, bạn sẽ tự tay dựng Jenkins bằng Docker, nối webhook, và viết một Jenkinsfile hoàn chỉnh deploy lên Kubernetes.

Phần II · Bài giải

Bài giải — Lab Session 2: Pod & Deployment

Lời giải chi tiết: minikube start, chạy FE/BE bằng Pod, chuyển sang Deployment.

Chuẩn bị

Đảm bảo đã cài `docker`, `minikube` và `kubectl`. Kiểm tra nhanh:

```
docker version
minikube version
kubectl version --client
```

Nếu cả ba lệnh chạy được, bắt đầu lab.

Lab 2.1 — Khởi động cluster và kiểm tra node

Bước 1. Khởi động minikube với driver Docker:

```
minikube start --driver=docker
```

Bước 2. Kiểm tra node:

```
kubectl get nodes
```

Kết quả mong đợi:

NAME	STATUS	ROLES	AGE	VERSION
minikube	Ready	control-plane	1m	v1.30.0

Giải thích output:

Cột	Ý nghĩa
NAME	Tên node (mặc định trùng tên cluster: <code>minikube</code>).
STATUS	<code>Ready</code> = node đã sẵn sàng nhận Pod. Nếu là <code>NotReady</code> , chờ thêm hoặc xem <code>minikube status</code> .
ROLES	<code>control-plane</code> — vì minikube gộp control plane và worker vào một node.
VERSION	Phiên bản Kubernetes đang chạy.

Vì sao chỉ có 1 node? minikube là môi trường học/dev. Một node đóng cả hai vai trò để nhẹ máy. Production sẽ có nhiều worker node tách biệt.

Lab 2.2 — Chạy FE và BE bằng Pod trần

Bước 1. Tạo `pod-backend.yaml` (Backend Go):

```
apiVersion: v1
kind: Pod
metadata:
  name: backend
  labels:
    app: backend
spec:
  containers:
    - name: backend
      image: golang:1.22
      command: ["sh", "-c", "echo BE up; sleep 3600"]
      ports:
        - containerPort: 8080
```

Vì sao có `command` ? Image `golang` gốc không tự chạy server nào cả — nếu không cho lệnh, container sẽ thoát ngay và Pod rơi vào `CrashLoopBackOff` . Ở đây ta cho nó “sống” để quan sát. App Go thật sẽ dùng image bạn tự build với binary làm entrypoint.

Bước 2. Tạo `pod-frontend.yaml` (Frontend demo bằng nginx):

```

apiVersion: v1
kind: Pod
metadata:
  name: frontend
  labels:
    app: frontend
spec:
  containers:
    - name: frontend
      image: nginx:1.27
      ports:
        - containerPort: 80

```

Bước 3. Apply cả hai:

```

kubectl apply -f pod-backend.yaml
kubectl apply -f pod-frontend.yaml
kubectl get pods

```

Kết quả:

NAME	READY	STATUS	RESTARTS	AGE
backend	1/1	Running	0	20s
frontend	1/1	Running	0	18s

Bước 4. Kiểm tra logs và describe:

```

kubectl logs backend          # thấy dòng "BE up"
kubectl describe pod frontend # xem phần Events ở cuối

```

Vì sao xem `describe` ? Nếu Pod không `Running` , phần **Events** cho biết chính xác nguyên ở đâu: kéo image lỗi (`ImagePullBackOff`), chưa gán được node, hay container crash. Đây luôn là bước debug đầu tiên.

Nhận xét vấn đề: Thử xóa một Pod và xem điều gì xảy ra:

```

kubectl delete pod frontend
kubectl get pods

```

frontend biến mất và **không có gì tạo lại**. Đây chính là lý do ta cần Deployment ở lab tiếp theo.

Lab 2.3 — Chuyển Pod thành Deployment, scale và rollout

Bước 1. Dọn Pod trần cũ:

```
kubectl delete pod backend --ignore-not-found
```

Bước 2. Tạo deployment-backend.yaml đầy đủ:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
  labels:
    app: backend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: golang:1.22
          command: ["sh", "-c", "echo BE up; sleep 3600"]
          ports:
            - containerPort: 8080
```

Vì sao selector.matchLabels phải khớp template.metadata.labels ? Đó là sợi dây để ReplicaSet nhận biết Pod nào thuộc quyền quản của mình. Nếu hai chỗ này lệch nhau, kubectl apply sẽ báo lỗi selector does not match template labels .

Bước 3. Apply và xem cả 3 tầng:

```
kubectl apply -f deployment-backend.yaml
kubectl get deploy,rs,pods
```

NAME	READY	UP-TO-DATE	AVAILABLE	AGE
deployment.apps/backend	1/1	1	1	10s

NAME	DESIRED	CURRENT	READY	AGE
replicaset.apps/backend-6f8c ...	1	1	1	10s

NAME	READY	STATUS	RESTARTS	AGE
pod/backend-6f8c ... -abcde	1/1	Running	0	10s

Vì sao thấy cả ReplicaSet dù ta chỉ tạo Deployment? Vì Deployment **tự sinh** ReplicaSet để quản Pod. Bạn không viết ReplicaSet bằng tay bao giờ.

Bước 4. Kiểm chứng khả năng tự hồi sinh — xóa Pod:

```
kubectl delete pod -l app=backend
kubectl get pods
```

Lần này một Pod mới **được tạo lại ngay** — vì ReplicaSet luôn giữ đủ `replicas`. Đây là điểm khác biệt cốt lõi so với Pod trần.

Bước 5. Scale lên 3 bản sao:

```
kubectl scale deployment/backend --replicas=3
kubectl get pods -l app=backend
```

Bây giờ có 3 Pod cùng chạy.

Vì sao scale dễ vậy? Ta chỉ đổi *trạng thái mong muốn* (`replicas=3`); ReplicaSet tự tạo thêm cho đủ. Với Pod trần bạn phải viết tay 3 file khác tên.

Bước 6. Rolling update — đổi image sang phiên bản mới:

```
kubectl set image deployment/backend backend=golang:1.23
kubectl rollout status deployment/backend
```

`rollout status` sẽ chờ đến khi tất cả Pod mới `Ready`. Trong lúc đó Deployment tăng dần Pod mới và giảm dần Pod cũ nên dịch vụ không đứt.

Bước 7. Xem lịch sử và rollback:

```
kubectl rollout history deployment/backend
kubectl rollout undo deployment/backend
kubectl rollout status deployment/backend
```

Vì sao rollback tức thì? ReplicaSet của phiên bản cũ chỉ bị scale về 0 chứ không bị xóa. `rollout undo` chỉ việc scale nó trở lại — không cần build hay kéo image lại từ đầu.

Kết quả đạt được

1. Dựng được cluster 1 node bằng `minikube start` và đọc hiểu `kubectl get nodes`.
2. Chạy FE/BE bằng Pod trần, dùng `logs` và `describe` để debug, và thấy rõ Pod **không tự hồi sinh**.
3. Chuyển sang Deployment: có tự hồi sinh, scale dễ dàng, rolling update và rollback an toàn.

Dọn dẹp khi học xong:

```
kubectl delete -f deployment-backend.yaml
kubectl delete pod frontend --ignore-not-found
minikube stop
```

Bài giải — Lab Session 3: Networking & RBAC

Lời giải: Service ClusterIP/NodePort, Namespace dev/staging, cấu hình RBAC.

Chuẩn bị

Đảm bảo cluster đang chạy (ví dụ minikube):

```
minikube start
kubectl get nodes
```

Lab 3.1 — Service ClusterIP cho BE + FE gọi qua DNS

Mục tiêu: backend chỉ truy cập nội bộ (ClusterIP), frontend gọi backend qua tên DNS, và expose frontend ra ngoài bằng NodePort.

Bước 1 — Deploy backend

Tạo file `backend.yaml` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
        - name: backend
          image: hashicorp/http-echo
          args: ["-text=hello from backend", "-listen=:8080"]
          ports:
            - containerPort: 8080
---
apiVersion: v1
kind: Service
metadata:
  name: backend-svc
spec:
  type: ClusterIP
  selector:
    app: backend
  ports:
    - port: 8080
      targetPort: 8080

```

```

kubectl apply -f backend.yaml
kubectl get svc backend-svc

```

Vì sao ClusterIP: backend không cần lộ ra ngoài. selector: app=backend giúp Service tự bám vào đúng nhóm Pod dù Pod đổi IP.

Bước 2 — Deploy frontend và test gọi backend qua DNS

Tạo file `frontend.yaml` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend
spec:
  replicas: 1
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
        - name: frontend
          image: nginx:alpine
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: frontend-svc
spec:
  type: NodePort
  selector:
    app: frontend
  ports:
    - port: 80
      targetPort: 80
      nodePort: 30080

```

```
kubectl apply -f frontend.yaml
```

Bước 3 — Test bằng curl từ trong pod

```
# Vào trong pod frontend
kubectl exec -it deploy/frontend -- sh

# Trong pod, gọi backend qua TÊN DNS, không dùng IP:
wget -qO- http://backend-svc:8080
# hoặc nếu có curl:
curl http://backend-svc:8080
# => hello from backend
```

Vì sao gọi được bằng `backend-svc` : CoreDNS phân giải `backend-svc` → ClusterIP của Service, vì frontend và backend cùng namespace `default` . Không cần biết IP thật.

Bước 4 — Expose frontend ra ngoài

```
# Cách 1: dùng NodePort đã khai báo (30080)
minikube ip          # lấy IP node
curl http://$(minikube ip):30080

# Cách 2: tiện hơn, minikube mở trình duyệt/URL giúp
minikube service frontend-svc
```

Vì sao dùng `minikube service` : trên minikube, NodePort đôi khi không truy cập trực tiếp được từ host; lệnh này tạo đường hầm và trả về URL truy cập thực tế.

Lab 3.2 — Namespace dev và staging

Mục tiêu: deploy cùng một app vào hai namespace tách biệt.

Bước 1 — Tạo hai namespace

```
kubectl create namespace dev
kubectl create namespace staging
kubectl get namespaces
```

Bước 2 — Deploy cùng app vào cả hai

Dùng lại `backend.yaml` nhưng apply vào từng namespace:

```
kubectl apply -f backend.yaml -n dev
kubectl apply -f backend.yaml -n staging
```

Bước 3 — So sánh

```
kubectl get pods,svc -n dev
kubectl get pods,svc -n staging
```

Kết quả: cả hai namespace đều có Deployment `backend` và Service `backend-svc` **trùng tên mà không xung đột**.

Vì sao không đụng nhau: object được đặt tên trong phạm vi namespace. Tên DNS đầy đủ cũng khác nhau: `backend-svc.dev.svc.cluster.local` và `backend-svc.staging.svc.cluster.local`. Muốn gọi chéo namespace phải dùng tên đầy đủ này.

Lab 3.3 — RBAC: chặn deploy chéo môi trường

Mục tiêu: tạo ServiceAccount `dev-user` chỉ được `get / list` trong namespace `dev`, và chứng minh KHÔNG deploy được lên `staging`.

Bước 1 — Tạo ServiceAccount trong dev

```
kubectl create serviceaccount dev-user -n dev
```

Bước 2 — Tạo Role chỉ đọc trong dev

Tạo file `dev-role.yaml`:

```

apiVersion: rbac.authorization.k8s.io/v1
kind: Role
metadata:
  namespace: dev
  name: dev-readonly
rules:
- apiGroups: ["", "apps"]
  resources: ["pods", "deployments", "services"]
  verbs: ["get", "list"]

```

```
kubectl apply -f dev-role.yaml
```

Vì sao chỉ `get / list` : đây là least privilege — `dev-user` được xem tài nguyên trong `dev` nhưng không được tạo/sửa/xóa. Không có `create` nghĩa là không deploy được.

Bước 3 — Gán Role bằng RoleBinding (chỉ trong dev)

Tạo file `dev-rolebinding.yaml` :

```

apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
  namespace: dev
  name: dev-readonly-binding
subjects:
- kind: ServiceAccount
  name: dev-user
  namespace: dev
roleRef:
  kind: Role
  name: dev-readonly
  apiGroup: rbac.authorization.k8s.io

```

```
kubectl apply -f dev-rolebinding.yaml
```

Vì sao RoleBinding (không phải ClusterRoleBinding): RoleBinding giới hạn hiệu lực trong namespace `dev` . Chủ thể không nhận được bất kỳ quyền nào ở namespace khác, kể cả `staging` .

Bước 4 — Chứng minh bằng kubectl auth can-i

```
# Trong dev: ĐƯỢC list pods ⇒ yes
kubectl auth can-i list pods \
  -n dev --as=system:serviceaccount:dev:dev-user
# ⇒ yes

# Trong dev: KHÔNG được tạo deployment ⇒ no
kubectl auth can-i create deployments \
  -n dev --as=system:serviceaccount:dev:dev-user
# ⇒ no

# Trong staging: KHÔNG được làm gì ⇒ no
kubectl auth can-i create deployments \
  -n staging --as=system:serviceaccount:dev:dev-user
# ⇒ no
kubectl auth can-i list pods \
  -n staging --as=system:serviceaccount:dev:dev-user
# ⇒ no
```

Bước 5 — Thử deploy thật để xác nhận bị chặn

```
kubectl apply -f backend.yaml \
  -n staging --as=system:serviceaccount:dev:dev-user
# ⇒ Error from server (Forbidden): deployments.apps is forbidden:
#   User "system:serviceaccount:dev:dev-user" cannot create resource
#   "deployments" in API group "apps" in the namespace "staging"
```

Vì sao bị Forbidden: API server kiểm tra RBAC trước khi thực hiện. dev-user không có RoleBinding nào trong staging, cũng không có create ở đâu cả, nên bị từ chối. Đây chính là least privilege hoạt động đúng.

Dọn dẹp

```
kubectl delete -f frontend.yaml -f backend.yaml
kubectl delete namespace dev staging
```

Tổng kết Lab

1. **3.1**: backend ClusterIP + frontend gọi qua DNS `backend-svc:8080` , expose bằng NodePort/ `minikube service` .
2. **3.2**: cùng app sống song song trong `dev` và `staging` nhờ cô lập namespace.
3. **3.3**: Role + RoleBinding giới hạn `dev-user` chỉ đọc trong `dev` , chứng minh không deploy được lên `staging` bằng `kubectl auth can-i` và lỗi Forbidden thực tế.

Bài giải — Lab Session 4: Go Deployment 3 replicas + probes

Lời giải: Deployment cho app Go với 3 replicas, livenessProbe và readinessProbe.

Bài giải — Lab Session 4: Go Deployment 3 replicas + probes

Mục tiêu: xây một ứng dụng Go nhỏ có hai endpoint sức khỏe (`/healthz` cho liveness, `/ready` cho readiness), đóng gói Docker, deploy lên Kubernetes với 3 replicas kèm probe và resources đầy đủ, rồi kiểm chứng self-healing.



Bước 1 — Viết app Go với `/healthz` và `/ready`

Tạo file `main.go`. App có một endpoint chính `/` và hai endpoint sức khỏe. Điểm tính tế: `/ready` chỉ trả `200` sau khi app “khởi động xong” (giả lập nạp cache mất vài giây).

```

package main

import (
    "fmt"
    "log"
    "net/http"
    "sync/atomic"
    "time"
)

// ready = 0 khi chưa sẵn sàng, = 1 khi đã sẵn sàng.
var ready int32

func main() {
    // Giả lập thời gian khởi động: nạp config/cache mất 5 giây.
    go func() {
        time.Sleep(5 * time.Second)
        atomic.StoreInt32(&ready, 1)
        log.Println("App đã sẵn sàng nhận traffic")
    }()

    http.HandleFunc("/", func(w http.ResponseWriter, r *http.Request) {
        fmt.Fprintln(w, "Xin chào từ go-api!")
    })

    // Liveness: chỉ cần tiến trình còn chạy và HTTP còn phản hồi.
    http.HandleFunc("/healthz", func(w http.ResponseWriter, r
    *http.Request) {
        w.WriteHeader(http.StatusOK)
        fmt.Fprintln(w, "ok")
    })

    // Readiness: chỉ 200 khi đã khởi động xong; ngược lại trả 503.
    http.HandleFunc("/ready", func(w http.ResponseWriter, r *http.Request)
    {
        if atomic.LoadInt32(&ready) == 1 {
            w.WriteHeader(http.StatusOK)
            fmt.Fprintln(w, "ready")
            return
        }
        w.WriteHeader(http.StatusServiceUnavailable)
        fmt.Fprintln(w, "not ready")
    })

    log.Println("Nghe trên cổng 8080")
    log.Fatal(http.ListenAndServe(":8080", nil))
}

```

Vì sao tách `/healthz` và `/ready` : `/healthz` (liveness) luôn `200` khi tiến trình còn sống — restart chỉ nên xảy ra khi app thật sự chết. `/ready` (readiness) trả `503` trong 5 giây đầu để Kubernetes chưa gửi traffic vào Pod chưa nạp xong. Đây chính là cơ chế giúp rolling update không rớt request.

Bước 2 — Dockerfile build image

Dùng multi-stage build cho image nhỏ gọn.

```
# Giai đoạn build
FROM golang:1.22 AS builder
WORKDIR /app
COPY main.go .
RUN go mod init go-api && CGO_ENABLED=0 go build -o /go-api main.go

# Giai đoạn runtime - image tối giản
FROM gcr.io/distroless/static-debian12
COPY --from=builder /go-api /go-api
EXPOSE 8080
ENTRYPOINT ["/go-api"]
```

Build và (nếu dùng Minikube) nạp image vào cluster:

```
docker build -t go-api:1.0.0 .
minikube image load go-api:1.0.0
```

Vì sao multi-stage + distroless: image cuối chỉ chứa binary tĩnh, không có shell hay package thừa → nhỏ, khởi động nhanh, và **giảm bề mặt tấn công** đáng kể so với image `golang` đầy đủ.

Bước 3 — Deployment YAML đầy đủ + Service

Tạo file `deploy.yaml` :

```

apiVersion: apps/v1
kind: Deployment
metadata:
  name: go-api
  labels:
    app: go-api
spec:
  replicas: 3
  selector:
    matchLabels:
      app: go-api
  template:
    metadata:
      labels:
        app: go-api
    spec:
      containers:
        - name: go-api
          image: go-api:1.0.0
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8080
          livenessProbe:
            httpGet:
              path: /healthz
              port: 8080
            initialDelaySeconds: 5
            periodSeconds: 10
            timeoutSeconds: 2
            failureThreshold: 3
          readinessProbe:
            httpGet:
              path: /ready
              port: 8080
            initialDelaySeconds: 3
            periodSeconds: 5
            timeoutSeconds: 2
            failureThreshold: 3
          resources:
            requests:
              cpu: "100m"
              memory: "64Mi"
            limits:
              cpu: "500m"
              memory: "128Mi"
---
apiVersion: v1
kind: Service

```

```
metadata:
  name: go-api
spec:
  selector:
    app: go-api
  ports:
    - port: 80
      targetPort: 8080
  type: ClusterIP
```

Lưu ý: `Service.selector` (`app: go-api`) phải khớp `labels` của Pod template, nếu không Service sẽ không tìm thấy Pod nào.

Bước 4 — Giải thích từng trường trong probe và VÌ SAO chọn giá trị đó

Trường	Giá trị	Vì sao
<code>initialDelaySeconds</code> (liveness)	5	Chờ 5s trước khi bắt đầu kiểm tra, để app kịp mở cổng 8080. Đặt quá thấp → app bị coi là “chết” ngay khi đang boot → restart vô hạn.
<code>periodSeconds</code>	10 (liveness) / 5 (readiness)	Tần suất kiểm tra. Readiness kiểm tra dày hơn để phát hiện Pod sẵn sàng sớm , rút ngắn thời gian rolling update.
<code>timeoutSeconds</code>	2	Nếu probe không phản hồi trong 2s coi là fail. Đủ rộng cho một endpoint HTTP đơn giản, không quá dài gây phát hiện chậm.
<code>failureThreshold</code>	3	Phải fail 3 lần liên tiếp mới hành động. Tránh phản ứng thái quá với một trục trặc thoáng qua (network glitch).
<code>readinessProbe.initialDelaySeconds</code>	3	Nhỏ hơn liveness vì logic app: readiness sẽ tự trả 503 trong 5s đầu, ta muốn K8s bắt đầu hỏi sớm để chuyển sang ready ngay khi app xong.

Vì sao readiness `initialDelaySeconds` (3) nhỏ hơn thời gian khởi động (5s): ta CỐ Ý để readiness bắt đầu hỏi trước khi app ready. Trong khoảng đó app trả 503, K8s giữ Pod ngoài Service. Đến giây thứ 5, `/ready` chuyển 200, Pod được thêm vào endpoints. Nếu bỏ readiness đi, traffic sẽ vào ngay giây đầu và user gặp lỗi.

Vì sao liveness KHÔNG trả `/ready`: nếu liveness dùng `/ready`, trong 5s khởi động app trả 503, liveness fail 3 lần và **restart Pod** — Pod không bao giờ lên được (CrashLoopBackOff). Đây là lỗi kinh điển cần tránh.

Bước 5 — Apply và theo dõi

```
kubectl apply -f deploy.yaml
kubectl get pods -w
```

Quan sát: 3 Pod lần lượt chuyển `ContainerCreating` → `Running`. Cột `READY` ban đầu là `0/1` (đang trong 5s khởi động, readiness chưa pass), sau vài giây chuyển `1/1`.

Kiểm tra Service đã gom đủ endpoints:

```
kubectl get endpoints go-api
```

Bạn phải thấy **3 địa chỉ IP** (3 Pod) trong danh sách endpoints.

Bước 6 — Kiểm chứng self-healing

Xóa thử một Pod và xem Deployment tự tạo lại:

```
# Lấy tên một Pod
kubectl get pods

# Xóa một Pod bất kỳ
kubectl delete pod <ten-pod>

# Theo dõi ngay
kubectl get pods -w
```

Quan sát: ngay khi một Pod chuyển `Terminating`, một Pod **mới** lập tức xuất hiện ở trạng thái `ContainerCreating`. Số lượng luôn quay về **3 replicas**.

Vì sao xảy ra điều này: Deployment (qua ReplicaSet) chạy vòng lặp điều khiển liên tục so sánh desired (3) với actual. Khi actual tụt xuống 2, nó tạo ngay Pod thứ ba để khớp lại. Ta không hề ra lệnh "tạo Pod mới" — hệ thống tự làm. Đó chính là self-healing.

Kiểm tra thêm rằng trong lúc một Pod bị xóa, Service vẫn phục vụ nhờ 2 Pod còn lại (readiness đã gỡ Pod đang chết khỏi endpoints):

```
kubectl get endpoints go-api -w
```

Tổng kết bài lab

1. App Go có `/healthz` (liveness, luôn 200 khi sống) và `/ready` (readiness, 503 trong lúc khởi động).
2. Dockerfile multi-stage → image nhỏ, an toàn.
3. Deployment 3 replicas với liveness/readiness và requests/limits.
4. `kubectl get pods -w` cho thấy vòng đời Pod và cột `READY` phản ánh readinessProbe.
5. `kubectl delete pod` chứng minh self-healing: cụm luôn duy trì 3 replicas.

Điểm cốt lõi cần nhớ: **liveness = restart khi chết, readiness = gỡ khỏi Service khi chưa/không sẵn sàng**. Cấu hình đúng hai probe này là ranh giới giữa một deployment “chạy tạm” và một deployment production thật sự chịu được sự cố.

Bài giải — Lab Session 5: Jenkins Pipeline & AI script

Lời giải: Jenkins bằng Docker, webhook, Jenkinsfile Build–Test–Deploy, shell script.

Lab 5.1 — Chạy Jenkins bằng Docker

Mục tiêu: dựng Jenkins trong container, giữ dữ liệu qua volume, truy cập tại cổng 8080.

1. **Tạo một named volume** để lưu dữ liệu Jenkins bền vững:

```
docker volume create jenkins_home
```

Vì sao: container là ephemeral — xóa container là mất sạch job, plugin, cấu hình. Volume `jenkins_home` sống độc lập với vòng đời container, nên dựng lại container vẫn còn nguyên dữ liệu.

2. **Chạy Jenkins** với volume gắn vào `/var/jenkins_home` và map cổng 8080:

```
docker run -d \
  --name jenkins \
  -p 8080:8080 \
  -p 50000:50000 \
  -v jenkins_home:/var/jenkins_home \
  jenkins/jenkins:lts-jdk17
```

Giải thích các cờ:

Cờ	Ý nghĩa
<code>-d</code>	chạy nền (detached)
<code>--name jenkins</code>	đặt tên container cho dễ quản lý
<code>-p 8080:8080</code>	cổng giao diện web
<code>-p 50000:50000</code>	cổng cho agent kết nối vào
<code>-v jenkins_home:/var/jenkins_home</code>	gắn volume giữ dữ liệu

3. Lấy mật khẩu mở khóa lần đầu (initial admin password):

```
docker exec jenkins cat /var/jenkins_home/secrets/initialAdminPassword
```

4. Mở trình duyệt tại `http://localhost:8080`, dán mật khẩu vừa lấy để **unlock**.

5. Chọn **Install suggested plugins** để cài bộ plugin phổ biến (Git, Pipeline...). Sau đó tạo tài khoản admin đầu tiên.

Vì sao dùng tag `lts-jdk17` thay vì `latest` : `lts` (Long-Term Support) ổn định cho production; ghim JDK giúp môi trường tái lập chính xác, tránh "bản latest tự đổi làm hỏng build".

Lab 5.2 — Webhook GitHub → Jenkins

Mục tiêu: mỗi khi có commit push lên GitHub, Jenkins tự chạy build.

- Trong Jenkins, tạo một **Pipeline job**, ở phần **Build Triggers** tick chọn **GitHub hook trigger for GITScm polling**.
- Trên GitHub, vào **Settings** → **Webhooks** → **Add webhook** của repo:
 - Payload URL:** `http://<JENKINS_PUBLIC_URL>:8080/github-webhook/`
 - Content type:** `application/json`
 - Which events:** chọn *Just the push event*.
- Bấm **Add webhook**. GitHub sẽ gửi một ping thử; nếu thấy dấu tích xanh là kết nối thành công.

Vì sao dùng webhook thay vì polling: webhook là cơ chế *push* — GitHub chủ động gọi Jenkins ngay khi có commit, build khởi động tức thì. Polling thì Jenkins phải hỏi Git định kỳ, vừa trễ vừa tốn tài nguyên.

Trường hợp không có GitHub (Git server tự dựng): dùng server-side hook `post-receive` để gọi Jenkins. Tạo file `hooks/post-receive` trên bare repo:

```
#!/bin/sh
# post-receive hook: báo Jenkins build khi nhận push
curl -X POST http://localhost:8080/job/my-app/build \
  --user "admin:${JENKINS_API_TOKEN}"
```

Vì sao đặt ở server-side: hook `post-receive` chạy trên Git server sau khi nhận push, dev không thể bỏ qua (khác client-side hook). Đây là điểm trigger đáng tin cậy.

Lab 5.3 — Jenkinsfile declarative đầy đủ

Mục tiêu: pipeline Checkout → Build → Unit Test → Build Image → Deploy lên Kubernetes. Đặt file tên `Jenkinsfile` ở gốc repo.

```

pipeline {
    agent any

    environment {
        IMAGE_NAME = "my-registry.example.com/my-app"
        IMAGE_TAG  = "${env.BUILD_NUMBER}"
        REGISTRY_CRED = credentials('docker-registry-cred')
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }

        stage('Build') {
            steps {
                echo "Đang build ứng dụng ..."
                sh 'make build'
            }
        }

        stage('Unit Test') {
            steps {
                echo "Đang chạy unit test ..."
                sh 'make test'
            }
        }

        stage('Build Image') {
            steps {
                sh "docker build -t ${IMAGE_NAME}:${IMAGE_TAG} ."
                sh "echo ${REGISTRY_CRED_PSW} | docker login my-
registry.example.com -u ${REGISTRY_CRED_USR} --password-stdin"
                sh "docker push ${IMAGE_NAME}:${IMAGE_TAG}"
            }
        }

        stage('Deploy') {
            steps {
                sh "kubectl set image deployment/my-app my-
app=${IMAGE_NAME}:${IMAGE_TAG}"
                sh "kubectl rollout status deployment/my-app"
            }
        }
    }
}

```

```

post {
  success {
    echo "Pipeline thành công – đã deploy ${IMAGE_TAG}"
  }
  failure {
    echo "Pipeline thất bại – không có gì được deploy."
  }
}
}

```

Điểm mấu chốt:

- `checkout scm` — lấy đúng commit đã trigger build.
- `IMAGE_TAG = "${env.BUILD_NUMBER}"` — mỗi build một tag duy nhất, không dùng `latest`.
Vì sao: tag cố định giúp truy vết chính xác image nào đang chạy và rollback dễ dàng.
- `credentials('docker-registry-cred')` tự tạo hai biến `_USR` và `_PSW`; dùng `--password-stdin` để **không lộ mật khẩu** trong log.
- `kubectl set image` cập nhật image của deployment, Kubernetes tự rolling update; `rollout status` chờ và xác nhận deploy xong.
- Block `post` báo kết quả — nếu test fail, các stage sau **không chạy**, không image nào bị deploy.

Vì sao đặt Jenkinsfile trong repo: pipeline tiến hóa cùng code, được review qua Pull Request và có đầy đủ lịch sử — khác hẳn cấu hình bấm tay trên UI.

Lab 5.4 — Shell script tự trigger build (AI gợi ý)

Mục tiêu: một git hook cục bộ gọi Jenkins API để kích hoạt build. Script dưới đây do AI gợi ý — **bạn phải review trước khi dùng**.

Tạo file `.git/hooks/post-commit` (client-side) hoặc dùng ở `post-receive` (server-side):

```
#!/usr/bin/env bash
set -euo pipefail

# Cấu hình lấy từ biến môi trường, KHÔNG hardcode token vào script
JENKINS_URL="${JENKINS_URL:?Chưa dat JENKINS_URL}"
JOB_NAME="${JENKINS_JOB:?Chưa dat JENKINS_JOB}"
JENKINS_USER="${JENKINS_USER:?Chưa dat JENKINS_USER}"
JENKINS_TOKEN="${JENKINS_TOKEN:?Chưa dat JENKINS_TOKEN}"

echo "Đang trigger build cho job: ${JOB_NAME}"

http_code=$(curl -sS -o /dev/null -w "%{http_code}" \
  -X POST "${JENKINS_URL}/job/${JOB_NAME}/build" \
  --user "${JENKINS_USER}:${JENKINS_TOKEN}")

if [ "${http_code}" = "201" ]; then
  echo "Đã trigger build thành công."
else
  echo "Lỗi khi trigger build. HTTP code: ${http_code}" >&2
  exit 1
fi
```

Nhớ cấp quyền chạy cho hook:

```
chmod +x .git/hooks/post-commit
```

Cảnh báo: luôn review script do AI sinh ra

Vì sao phải review: AI có thể gợi ý script *chạy được* nhưng không *an toàn*. Trước khi dùng bất kỳ script nào, hãy kiểm tra:

- **Không hardcode secret:** token phải lấy từ biến môi trường (như trên), tuyệt đối không viết thẳng vào script rồi commit — vào lịch sử Git là lộ vĩnh viễn.
- **Kiểm soát lỗi:** `set -euo pipefail` để script dừng khi có lỗi thay vì âm thầm chạy tiếp.
- **Xác thực URL/endpoint:** đảm bảo gọi đúng Jenkins của bạn, không phải một domain lạ AI vô tình chèn.
- **Quyền tối thiểu:** dùng Jenkins API token (tạo trong *User* → *Configure* → *API Token*) thay vì mật khẩu tài khoản, và token chỉ nên có quyền trigger job.
- **Đọc từng dòng:** không chạy lệnh mình chưa hiểu, đặc biệt các lệnh có `rm`, `curl` | `sh`, hay ghi đè file.

Nguyên tắc vàng: AI là trợ lý gợi ý, không phải người chịu trách nhiệm. Bạn là người ký duyệt cuối cùng cho mọi dòng code chạm tới hệ thống thật.

Tổng kết Lab

1. **5.1** — Dựng Jenkins bằng Docker, volume `jenkins_home` giữ dữ liệu, cổng 8080, unlock và cài plugin.
2. **5.2** — Nối webhook GitHub (hoặc `post-receive` hook) để commit tự trigger build.
3. **5.3** — Jenkinsfile declarative Checkout → Build → Test → Image → Deploy, dùng credentials và tag theo build number.
4. **5.4** — Git hook gọi Jenkins API bằng `curl`, secret lấy từ env, và luôn review kỹ script AI trước khi chạy.